UNIVERSITÀ DEGLI STUDI DI UDINE
HIC SUNT FUTURA

DIPARTIMENTO DI SCIENZE MATEMATICHE, INFORMATICHE E FISICHE

TESI DI LAUREA IN
INFORMATICA

RAM-USB: Progettazione, implementazione e distribuzione di un servizio di backup multi-utente secondo i principi zero-trust e defense-in-depth

CANDIDATO
Francesco Verrengia

RELATORE
Prof. Ivan Scagnetto

Anno accademico 2025-2026

Contatti dell'Istituto
Dipartimento di Scienze Matematiche, Informatiche e Fisiche
Università degli Studi di Udine
Via delle Scienze, 206
33100 Udine — Italia
+39 0432 558400
https://www.dmif.uniud.it/

© 2026 Francesco Verrengia
This work is shared under the Creative Commons 4.0 License Attribution-NonCommercial-ShareAlike.

To my best friend Marianna Massari.
The light may flicker,
the light may fade,
even the dark with you feels like the brightest day

Ringraziamenti

Ai miei genitori, per non aver mai dubitato di me, e per avermi dato l'occasione di sbagliare in mille modi e imparare in altri mille. Grazie per non aver mai provato a influenzare le mie scelte e per avermi sempre appoggiato (e sovvenzionato) per quanto folli queste potessero sembrare.

A mia sorella Chiara, per non aver mai smesso di avvicinarsi nonostante la mia apatia e per essere stata il mio punto di riferimento in tutti questi anni. Sappi che ti voglio bene, anche se cerco sempre di scappare dai tuoi abbracci.

Alla mia inarrestabile zia Elvira, che nonostante le mille cose da fare, non ha mai saltato un mio compleanno o un onomastico, ed è sempre stata presente, anche nei momenti più inaspettati.

Ai miei cugini, Antonio e Adriana, per aver cercato di avvicinarsi a me ogni volta che ne hanno avuto l'occasione e per aver chiesto il mio parere per ogni cosa in cui c'entrasse anche minimamente l'informatica.

A mia zia Orsola, che insiste sempre per farsi venire a trovare. Grazie per esserci sempre stata per me e per aver chiesto i miei consigli, nei momenti più critici.

A mio zio Franco, sappi che se sono arrivato fin qui, è anche merito dell'affetto che mi hai sempre dato. Spero tu sia orgoglioso di quello che sono oggi.

A mio cugino Alessandro, con la speranza che trovi la sua strada, come ho fatto io, fregandosene delle parole di chi non può capire. Fai quello che ti piace, coltiva le tue passioni, sbaglia, cresci e riprova. Ricorda che potrai sempre contare su di me.

A voi tutti dico grazie, per aver attraversato l'Italia solo per essere con me oggi.

Ai miei amici e compagni di studi: grazie, per aver condiviso con me le esperienze più belle degli ultimi quattro anni.

Primo tra tutti Ale, per tutte le passeggiate in centro, i gelati pieni di lattosio che non avrei dovuto mangiare, le serate giochi da tavolo e le lunghe e profonde chiacchierate nella Verremobile fino alle due del mattino. Per le flashcards, i pomodori e tutte le tecniche di studio che ci hai tenuto a condividere con me. Spero ancora che tu condivida tutto il tuo lavoro con il resto della community, e che in futuro, da ricercatore, ti abituerai a condividere ciò che imparerai con il resto del mondo.

Ricky, a te più di tutti, devo molto. Grazie per aver appoggiato la mia idea partecipando al progetto di IoT, anche se con l'IoT aveva ben poco a che fare. Ad ogni modo grazie, perché da lì sono nati il mio interesse per l'Open Source, la passione per la progettazione, i miei lavori su GitHub e le contribuzioni a Signal e Lumo. Tutte cose di cui vado molto fiero. Infine grazie, perché questa tesi non esisterebbe senza di te.

Marty, grazie per essermi sempre stata vicina e per aver evitato che io andassi fuori di testa quando studiavamo Algoritmi. Grazie per le serate passate a guardare le live di Cydo, per avermi fatto conoscere e amare D&D e per la bellissima esperienza al Lucca Comics. Siamo letteralmente passati in un ciclone e quasi volati giù da una montagna quella sera, ma è stata un'esperienza incredibile che non dimenticherò. Hai un percorso difficile di fronte a te, ma non ho alcun dubbio che ce la farai.

Mattone, non dimenticherò mai il giorno in cui ci siamo conosciuti, nel laboratorio di architettura. Meriti un ringraziamento speciale perché senza di te, questo gruppo non si sarebbe mai formato. Ormai ci sei quasi, ti mancano solo 3 esami, anche se sono i più tosti della triennale. Non mollare, ma ti prego, cerca di fare un po' meno detox. Ora che sono un dottore posso dirti che il telefono fa male alla salute, soprattutto alla tua.

Tommy, grazie per le grigliate a ferragosto e per aver aggiunto un posto all'ultimo minuto per me. Spero che casa tua rimanga il punto di incontro del gruppo anche per i prossimi anni.

Infine, ringrazio Marianna Massari, senza la quale oggi, sicuramente non sarei qui. Grazie per aver fatto nascere in me la passione per l'informatica, quel giorno in cui, solo per giocare con te, ho assemblato il mio primo PC. Grazie per quel 6 dicembre 2022, quando hai portato la luce nel mio momento più buio. Grazie, per essere stata dalla mia parte sempre, anche quando io stesso non riuscivo ad esserlo. Grazie, per le oltre 500 canzoni che mi hai regalato, e che ho ascoltato e amato ogni giorno, negli ultimi 9 anni. Infine, grazie per avermi dato un nome, con cui firmo orgoglioso ogni mio progetto.

Sommario

La tesi presenta la progettazione, l'implementazione e la distribuzione di RAM-USB, un servizio di backup multi-utente accessibile da remoto, progettato secondo i principi zero-trust e defense-in-depth.

La metodologia adottata combina il modello a spirale per la definizione e l'evoluzione dei requisiti, l'approccio Test-Driven Development per l'implementazione e il modello a V per la loro verifica. Garantisce, inoltre, la tracciabilità monodirezionale dal requisito al codice e bidirezionale dal requisito al commit e dal requisito al test.

Il lavoro approfondisce in particolare la progettazione dei servizi di autenticazione e controllo dinamico degli accessi, nonché le soluzioni adottate per la protezione delle credenziali degli utenti, analizzandone criticamente vantaggi, svantaggi e implicazioni in termini di sicurezza. Il sistema che li realizza è un'architettura client-server a microservizi n-tier, articolata in componenti dedicati all'accesso, alla registrazione, alla gestione dei backup e all'osservabilità delle metriche, distribuiti in container Docker a loro volta isolati in Kernel-based Virtual Machines e Linux Containers su uno o più host Proxmox.

Le comunicazioni interne avvengono tramite API REST per le operazioni di registrazione e login, e tramite MQTT per la pubblicazione delle metriche operative; tutte restano confinate alla rete mesh coordinata da Headscale, autenticate reciprocamente via mTLS con certificati X.509 emessi da una Certificate-Authority interna.

In accordo con il principio zero-trust, ogni componente rivalida il formato e il contenuto di ogni richiesta, senza dare per scontato che il componente precedente lo abbia già fatto. A seguito dell'autenticazione e dell'autorizzazione, il sistema modifica dinamicamente le regole di accesso della rete mesh, consentendo all'utente di raggiungere il proprio spazio di backup tramite SFTP, isolato da quello degli altri utenti mediante chroot. I file dei backup vengono deduplicati, compressi e cifrati tramite Restic prima di lasciare il dispositivo dell'utente, e la loro riservatezza resta quindi garantita dalla cifratura lato client.

La tesi si conclude con una valutazione critica delle proprietà di sicurezza, riservatezza e funzionalità del sistema nello scenario di compromissione totale di un singolo servizio. L'analisi permette di valutare l'efficacia delle misure di isolamento e delle strategie di defense-in-depth adottate, evidenziandone al contempo i limiti e le proprietà che verrebbero preservate anche in presenza di una grave compromissione del servizio.

Indice

Elenco delle tabelle **xiii**

Elenco delle figure **xv**

Elenco degli acronimi **xix**

1 Introduzione **1**
1.1 Contesto . . . 1
1.1.1 Funzioni di hash . . . 2
1.2 Problema affrontato . . . 2
1.3 Obiettivi . . . 2
1.4 Metodologia . . . 2
1.5 Struttura della tesi . . . 3

2 Principi fondanti **5**
2.1 Zero-trust . . . 5
2.2 Defense-in-depth . . . 5

3 Requisiti: metodologia e casi d'uso **7**
3.1 Metodologia dei requisiti . . . 7
3.2 Tracciabilità dei requisiti . . . 7
3.3 Requisiti utente e casi d'uso . . . 8

4 Architettura generale e zone di fiducia **11**
4.1 Panoramica dell'architettura . . . 11
4.1.1 Entry-Hub . . . 11
4.1.2 Security-Switch . . . 12
4.1.3 Database-Vault . . . 12
4.1.4 Storage-Service . . . 13
4.1.5 Network-Manager . . . 13
4.1.6 Certificate-Authority . . . 13
4.1.7 Headscale . . . 14
4.1.8 Mosquitto . . . 14
4.1.9 Metrics-Collector . . . 14
4.1.10 Grafana . . . 14
4.2 Modello di comunicazione inter-servizio . . . 15
4.3 Zone di fiducia e confini di sicurezza . . . 16
4.4 PKI e gestione dei certificati . . . 17

5 Autenticazione e gestione delle identità **19**
5.1 Ciclo di vita della richiesta . . . 19
5.2 Cifratura e hashing delle credenziali . . . 20
5.2.1 Robustezza della password . . . 22
5.3 Registrazione . . . 24

5.4 Login . . . 25
5.5 Orchestrazione post-autenticazione . . . 27

6 Storage e backup dei file **29**
6.1 Modello di accesso SFTP-only . . . 29
6.2 Isolamento e chroot dinamico . . . 30
6.3 Backup e ripristino . . . 30

7 Rete e coordinamento della mesh **33**
7.1 Politiche di rete e confini di raggiungibilità . . . 33
7.2 Ciclo di vita dei permessi di accesso . . . 33
7.3 Il paradosso del coordinatore: Headscale come servizio a sé . . . 35
7.4 CG-NAT e la necessità di un endpoint pubblico dedicato . . . 36

8 Osservabilità e metriche **37**
8.1 Pubblicazione delle metriche . . . 37
8.2 Ricezione e controllo di coerenza . . . 38
8.3 Persistenza delle serie temporali . . . 38
8.4 Visualizzazione delle metriche . . . 38

9 Testing e validazione **41**
9.1 Il modello V e i quattro livelli di test . . . 41
9.2 Strategia di integrazione bottom-up . . . 41
9.3 TDD: criteri di ingresso e uscita . . . 42
9.4 Copertura e lacune del piano di test . . . 42
9.5 Verifica automatica pre-commit . . . 43
9.6 Risultati: verifiche completate . . . 43

10 Distribuzione **45**
10.1 Containerizzazione e supervisione s6-overlay . . . 45
10.2 Perché non tutti i container sono distroless . . . 45
10.3 Docker locale vs Proxmox VE: due livelli di isolamento . . . 46
10.4 Topologia distribuita . . . 47
10.5 Alta disponibilità e resilienza: stato attuale e limiti . . . 48

11 Analisi critica delle proprietà di sicurezza **49**
11.1 Scenari di attacco e mitigazioni . . . 49
11.1.1 Enumerazione degli indirizzi email in registrazione . . . 49
11.1.2 Accesso amministrativo alle credenziali . . . 50
11.1.3 Verifica offline di un indirizzo a partire da una copia del database . . . 50
11.2 Isolamento dei fallimenti . . . 51
11.2.1 Entry-Hub . . . 51
11.2.2 Certificate-Authority . . . 52
11.2.3 Database-Vault . . . 52
11.2.4 Network-Manager . . . 53
11.2.5 Storage-Service . . . 53

12 Conclusioni **55**
12.1 Risultati raggiunti . . . 55
12.2 Limiti noti . . . 55
12.3 Sviluppi futuri . . . 56

A Elenco completo dei requisiti **57**
A.1 Requisiti di sistema funzionali . . . 57
A.1.1 Client . . . 57
A.1.2 Entry-Hub . . . 58
A.1.3 Security-Switch . . . 59
A.1.4 Database-Vault . . . 60
A.1.5 Storage-Service . . . 61
A.1.6 Network-Manager . . . 62
A.1.7 Certificate-Authority . . . 64
A.1.8 Sistema di monitoraggio . . . 64
A.1.9 Infrastruttura di rete e PKI . . . 64
A.2 Requisiti non funzionali . . . 65
A.2.1 Di prodotto . . . 65
A.2.2 Organizzativi . . . 65
A.2.3 Esterni . . . 66
A.3 Requisiti di dominio . . . 66

Bibliografia **67**

Elenco delle tabelle

5.1 Codici HTTP restituiti al client e loro significato nel sistema. 20
5.2 Campi della stringa PHC in cui Database-Vault salva l'hash della password. 22

9.1 Livello di verifica associato a ciascun livello di specifica. 41
9.2 Proprietà verificate sul sistema in esecuzione. 44

10.1 Immagine di base per servizio e processi che condividono lo stesso container. 46

11.1 Requisiti Utente verificati in questa sezione contro la compromissione totale di un servizio. 51

A.1 Requisiti funzionali del Client. 57
A.2 Requisiti funzionali di Entry-Hub. 58
A.3 Requisiti funzionali di Security-Switch. 59
A.4 Requisiti funzionali di Database-Vault. 60
A.5 Requisiti funzionali di Storage-Service. 61
A.6 Requisiti funzionali di Network-Manager. 62
A.7 Requisiti funzionali di Certificate-Authority. 64
A.8 Requisiti funzionali del sistema di monitoraggio (MQTT-Broker / Metrics-Collector / TimescaleDB / Grafana). 64
A.9 Requisiti funzionali di rete e PKI. 64
A.10 Requisiti non funzionali di prodotto. 65
A.11 Requisiti non funzionali organizzativi. 65
A.12 Requisiti non funzionali esterni. 66
A.13 Requisiti di dominio (RD). 66

Elenco delle figure

3.1 Diagramma dei casi d'uso principali di RAM-USB. 9

4.1 Diagramma Entità-Relazioni del record utente in Database-Vault. 12
4.2 Struttura del filesystem di Storage-Service, da `creator.go` e `dirmaker.go` [1, 2]. 13
4.3 Diagramma dei container di RAM-USB. 15
4.4 Zone di fiducia e raggiungibilità sulla rete mesh. 16
4.5 Gerarchia delle Certificate Authority e relazioni di firma. 17

5.1 Sequenza di registrazione (UC-01). 25
5.2 Sequenza di autenticazione (UC-02). 26

6.1 Sequenza di backup di un file (UC-03). 31

7.1 Ciclo di vita del permesso di accesso associato al nodo di un utente. 35

8.1 Pipeline di osservabilità, dalla pubblicazione delle metriche alla dashboard di Grafana. . 39

10.1 Topologia di produzione target. 47

Elenco degli acronimi

Acronimo	Significato
ACL	Access Control List
AES-GCM	Advanced Encryption Standard, Galois/Counter Mode
API	Application Programming Interface
CA	Certificate Authority
CG-NAT	Carrier-Grade Network Address Translation
CLI	Command Line Interface
DNS	Domain Name System
GDPR	General Data Protection Regulation
HKDF	HMAC-based Key Derivation Function
HMAC	Hash-based Message Authentication Code
HTTP/HTTPS	HyperText Transfer Protocol (Secure)
IP	Internet Protocol
JSON	JavaScript Object Notation
KVM	Kernel-based Virtual Machine
LXC	Linux Container
MQTT	Message Queuing Telemetry Transport
mTLS	mutual Transport Layer Security
NAT	Network Address Translation
NIST	National Institute of Standards and Technology
OWASP	Open Web Application Security Project
PDF	Portable Document Format
PHC	Password Hashing Competition
PKI	Public Key Infrastructure
POSIX	Portable Operating System Interface
QoS	Quality of Service
RAID	Redundant Array of Inexpensive Disks
REST	REpresentational State Transfer
RFC	Request for Comments
SFTP	Secure Shell File Transfer Protocol
SHA	Secure Hash Algorithm
SRS	Software Requirements Specification
SSH	Secure Shell

Acronimo	**Significato**
TCP	Transmission Control Protocol
TDD	Test-Driven Development
TLS	Transport Layer Security
UID	User Identifier
VE	Virtual Environment
VPS	Virtual Private Server

1 Introduzione

1.1 Contesto

Un backup, nell'ambito della sicurezza informatica, è un processo di disaster recovery: un procedimento che consente di recuperare dati andati persi, per esempio in seguito a un attacco, un guasto hardware o software, o un errore umano. Una delle prime risposte tecniche alla perdita di dati è la ridondanza dei dischi su cui questi sono memorizzati: nel 1988 Patterson, Gibson e Katz formalizzano il RAID (Redundant Array of Inexpensive Disks) [3], che distribuisce lo stesso dato su più dischi economici per ottenere un'affidabilità paragonabile a quella di un singolo disco più costoso.

Questa ridondanza, però, resta locale e non protegge da un evento che distrugga l'intero apparato. Da qui nasce l'esigenza di un backup remoto: una copia dei dati conservata in un luogo diverso da quello degli originali. Il modo più semplice è affidarli a terzi, un conoscente che possiede l'hardware o un'attività commerciale che offre questo tipo di servizio, ma questo richiede di fidarsi che quei dati non vengano letti, modificati o venduti da chi li conserva.

Chi offre il servizio, dal canto suo, trova conveniente servire più utenti contemporaneamente. Questo richiede però di distinguere gli utenti, e di garantire che ciascuno acceda solo ai propri dati: in pratica, richiede un sistema di autenticazione e controllo degli accessi.

Tra i diversi sistemi di autenticazione esistenti, quello adottato e discusso in questa tesi è basato su email e password che, in quanto dati sensibili, richiedono un livello di protezione elevato. Il principio su cui si fonda questo tipo di autenticazione è semplice: l'utente si registra inviando al server le proprie credenziali; durante il login le invia di nuovo, e il server le confronta con quelle salvate in fase di registrazione. Se coincidono, l'utente è autenticato.

Il fatto che il server possieda le credenziali espone, però, tutti gli utenti al rischio che vengano rubate. Esistono numerosi sistemi per impedirlo; uno tra tutti, approfondito in questa tesi, è l'uso di una funzione di hash. Il server, al posto di salvare le credenziali in chiaro, le salva sotto forma di hash. Durante il login, ricalcola gli hash delle credenziali fornite dall'utente e verifica se coincidono con quelli salvati.

1.1.1 Funzioni di hash

Le funzioni di hash sono funzioni non invertibili che trasformano un input di lunghezza arbitraria in un output di lunghezza fissa e strettamente correlato con i dati di input, chiamato hash. Indicata con H una funzione di hash, questa gode delle seguenti proprietà fondamentali:

- **resistenza alla preimmagine:** dato h, è computazionalmente intrattabile trovare x tale che $H(x) = h$;
- **resistenza alla seconda preimmagine:** data x, è computazionalmente intrattabile trovare $y \neq x$ tale che $H(x) = H(y)$;
- **resistenza alle collisioni:** è computazionalmente intrattabile trovare una qualsiasi coppia (x, y) con $x \neq y$ tale che $H(x) = H(y)$.

1.2 Problema affrontato

Tra le diverse cause di perdita di dati, il progetto si concentra su quella derivante dall'attacco di un malintenzionato, adottando i principi di zero-trust e defense-in-depth per garantire la sicurezza dei dati anche in caso di compromissione di un componente qualsiasi del sistema.

1.3 Obiettivi

RAM-USB, Remotely Accessible Multi-User Secure Backup, è un caso di studio approfondito sulla progettazione, implementazione e distribuzione di un servizio di backup che soddisfi i requisiti descritti nel Capitolo 3. Non è pensato per competere con le soluzioni commerciali esistenti, ma per discuterne le scelte di design, illustrandone pro e contro in modo trasparente, con un approccio security first e concentrato sulla difesa del sistema da attacchi esterni.

1.4 Metodologia

Parte dei componenti di RAM-USB nascono dalla fase di brainstorming condotta insieme a Riccardo Gottardi [4].

La metodologia adottata combina il modello a spirale per la definizione e l'evoluzione dei requisiti [5] con l'approccio Test-Driven Development per l'implementazione e il modello a V per la loro verifica [6].

Il linguaggio usato per l'intero stack è Go, un linguaggio compilato progettato esplicitamente per gestire in modo efficiente molte connessioni di rete simultanee.

Per orchestrare l'infrastruttura attorno al codice Go (avvio dei servizi, supervisione dei processi nei container, sequenziamento all'avvio) è stato usato il linguaggio Bash.

Il codice è versionato con Git e pubblicato gratuitamente, in versione integrale su GitHub, distribuito con licenza MIT.

L'implementazione è stata interamente realizzata con l'assistenza dei Large Language Model Claude Sonnet 4.5 e Sonnet 5, in forma multi-agentica tramite l'harness Claude Code eseguito in una macchina virtuale su Proxmox per questioni di sicurezza e privacy.

L'intelligenza artificiale generativa è stata impiegata esclusivamente come strumento sotto stretta supervisione umana. La progettazione, la struttura del codice, i confini dei moduli, le firme, il comportamento delle singole funzioni, e i test corrispondenti restano una decisione umana; l'assistente ha implementato solo quanto specificato, in passi piccoli, incrementali e verificabili, prendendosi la libertà implementativa di basso livello. Ogni modifica è stata revisionata ed esplicitamente approvata prima di essere introdotta nella codebase. Le numerose strategie di mitigazione degli errori implementativi e allucinazioni dei due modelli, nonché la gestione dei multipli agenti utilizzati, non sono oggetto di questa tesi.

1.5 Struttura della tesi

Il Capitolo 2 approfondisce i due principi di sicurezza su cui RAM-USB è costruito, zero-trust e defense-in-depth.

Il Capitolo 3 descrive la metodologia con cui i requisiti sono stati raccolti e tracciati, e i casi d'uso che ne derivano.

Il Capitolo 4 presenta i componenti che partecipano alla registrazione e al login.

Il Capitolo 5 segue le credenziali lungo quel percorso, dal confine pubblico fino al record salvato, e descrive le regole che ogni strato applica.

Il Capitolo 6 segue il backup fino al proprio spazio isolato su Storage-Service.

Il Capitolo 7 descrive come la rete mesh applichi i confini di raggiungibilità tra i componenti.

Il Capitolo 8 chiude il ciclo di vita di una richiesta con la pipeline di osservabilità che la rende misurabile.

Il Capitolo 9 verifica il sistema così costruito attraverso i quattro livelli del modello a V, dal test di unità alla lista di accettazione, e riporta cosa ha trovato eseguendolo su hardware reale.

Il Capitolo 10 descrive come questi componenti siano effettivamente distribuiti, tra container Docker, macchine Proxmox e un VPS pubblico.

Il Capitolo 11 rilegge l'intero sistema dal punto di vista di chi tenta di comprometterlo, verificando quali garanzie resistano anche alla sua compromissione totale.

Il Capitolo 12 chiude la tesi bilanciando i risultati ottenuti con i limiti riscontrati, e indicando gli sviluppi futuri più rilevanti.

2

Principi fondanti

2.1 Zero-trust

Il termine ≪"zero trust"≫ venne coniato nel 2010 dall'analista Forrester John Kindervag. L'idea di fondo era quella di non concedere fiducia in base alla posizione di rete, ma verificare ogni accesso indipendentemente da essa.

Il NIST Special Publication 800-207 [7] formalizza oggi questo principio come l'assunzione che nessuna rete, interna o esterna, debba essere considerata implicitamente affidabile, e che ogni richiesta di accesso vada valutata indipendentemente dalla sua provenienza.

RAM-USB estende il principio zero-trust diffidando persino dell'identità di un componente autenticato via mTLS. Questo perché ogni componente suppone che gli altri siano sotto il controllo di un attaccante.

2.2 Defense-in-depth

Il NIST Special Publication 800-53 Revision 5 definisce la defense-in-depth come una strategia che integra persone, tecnologia e capacità operative per stabilire barriere ridondanti su più livelli di un'organizzazione, in modo che il guasto o l'aggiramento di una singola barriera non comprometta il sistema nel suo complesso [8].

3

Requisiti: metodologia e casi d'uso

3.1 Metodologia dei requisiti

I requisiti di RAM-USB sono raccolti in un documento unico chiamato Software Requirements Specification (SRS) [9], che costituisce l'unica fonte di verità su cosa il sistema debba fare. La scrittura di tale documento segue un modello a spirale dell'ingegneria del software [5], guidato dalla progressiva risoluzione del rischio, inteso come le fonti di incertezza più significative del progetto. Il livello di dettaglio dell'SRS aumenta di conseguenza ad ogni iterazione, perché, nonostante gli obiettivi e i requisiti di alto livello fossero chiari fin dalle fasi iniziali, la conoscenza degli strumenti e delle tecnologie di terze parti non era sufficiente a prevedere con certezza il comportamento a basso livello di questi strumenti. La scelta del modello a spirale nasce da questo squilibrio. Molti vincoli concreti sono infatti emersi solo nel momento dell'implementazione, quando l'interazione diretta con questi strumenti ha rivelato dettagli che una fase di analisi puramente teorica non è stata in grado di anticipare. Un caso concreto di questa dinamica è il requisito `NM-F-12`, rivisitato in tre round successivi:

nella versione 1.0 dell'SRS [10] assumeva che Network-Manager (il componente responsabile della rete privata del sistema) dovesse costruire da zero un proprio meccanismo per autorizzare l'accesso di nuovi dispositivi;

nella versione 1.4 [11] è emerso che Headscale, lo strumento di coordinamento della rete impiegato dal sistema, offriva già questa funzionalità tramite la propria CLI, eliminando la necessità di scriverla;

nella versione 1.7 [12] è emerso un limite più profondo dello stesso Headscale (non può coordinare una rete di cui fa parte) approfondito nella sezione 7.3, richiedendo una riformulazione più ampia del requisito.

3.2 Tracciabilità dei requisiti

Un aspetto metodologico distintivo del progetto è la tracciabilità, intesa come vincolo tecnico verificabile in tre direzioni indipendenti. Questo approccio privilegia la semplicità di un audit di sicurezza a scapito della complessità della gestione dell'implementazione.

- **Dal requisito alla sua implementazione**: ogni requisito dell'SRS che ha già un'implementazione è collegato a un permalink GitHub che punta a un commit specifico e ad un intervallo di righe preciso.
- **Dal test al requisito e viceversa**: ogni funzione di test porta, come commento di documentazione nella riga immediatamente precedente, l'identificativo del requisito che verifica. La relazione non è necessariamente 1-a-1 visto che un requisito può avere più test che lo verificano. Inoltre, la tracciabilità vale in entrambe le direzioni: si può infatti passare dal requisito ai test, cercando l'ID nella codebase; e dal test al requisito, leggendo il commento sovrastante la funzione.
- **Dal commit al requisito**: ogni commit del repository segue una grammatica fissa, descritta nel `CONTRIBUTING.md` [13] e qui sintetizzata come tipo, area, descrizione del cambiamento, seguiti dall'elenco degli identificativi dei requisiti che copre. Come per il collegamento test-requisito, anche questo vale in entrambe le direzioni: dal requisito, se ne cerca l'ID nella storia del repository; nella direzione opposta, l'elenco finale della descrizione di ogni commit indica quali requisiti sono coperti.

3.3 Requisiti utente e casi d'uso

I requisiti utente (RU) esprimono, dal punto di vista di chi usa il sistema, cosa RAM-USB deve garantire. Sono gli unici requisiti riportati integralmente nel corpo della tesi; i requisiti di sistema che li realizzano (funzionali, non funzionali e di dominio) sono troppo numerosi per un elenco completo, e sono raccolti in Appendice A.

RU-01 Registrarsi al servizio fornendo la quantità minima di dati necessaria.

RU-02 Potersi autenticare in sessioni successive alla registrazione.

RU-03 Poter caricare backup cifrati verso il servizio da qualsiasi luogo.

RU-04 Poter accedere al proprio backup solo dopo essersi autenticato.

RU-05 Avere la garanzia che nessuno, amministratori inclusi, possa leggere i propri backup.

RU-06 Avere la garanzia che nessuno, amministratori inclusi, possa leggere le proprie credenziali di accesso.

RU-07 Poter recuperare i propri file in qualsiasi momento.

RU-08 Avere la certezza che i propri dati siano isolati da quelli degli altri utenti.

RU-09 *(Fuori perimetro)* Avere la garanzia che nessuno, amministratori inclusi, possa modificare o cancellare i propri backup.

RU-10 (Amministratore) Poter osservare la salute e le prestazioni del sistema senza compromettere i dati degli utenti.

I requisiti utente si traducono nei seguenti casi d'uso, illustrati anche nella Figura 3.1.

UC-01 (Registrazione) Un nuovo utente fornisce email, password e chiave pubblica SSH, e ottiene un account attivo.

UC-02 (Autenticazione) Un utente registrato fornisce email e password, e ottiene un accesso temporaneo al proprio spazio di backup.

UC-03 (Backup di un file) Un utente autenticato carica un file, già cifrato sul proprio dispositivo, nel proprio spazio isolato.

UC-04 (Restore di un file) Un utente autenticato recupera un proprio file precedentemente caricato, decifrandolo localmente.

UC-05 (Consultazione delle metriche operative) Un amministratore consulta lo stato di salute e le prestazioni del sistema, senza accedere ai dati dei singoli utenti.

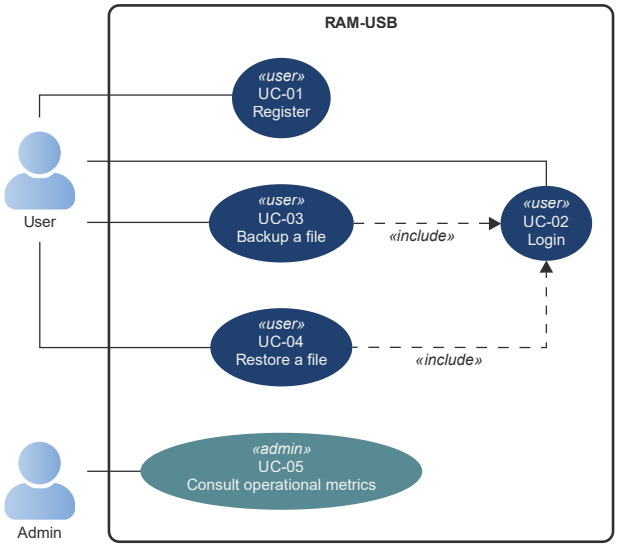


Figura 3.1: Diagramma dei casi d'uso principali di RAM-USB.

I capitoli successivi descrivono come l'architettura del sistema realizzi questi casi d'uso: il Capitolo 4 introduce la struttura generale dei componenti e i confini di sicurezza tra di essi, mentre i capitoli 5–8 approfondiscono ciascun dominio funzionale nel dettaglio.

4

Architettura generale e zone di fiducia

4.1 Panoramica dell'architettura

RAM-USB è un'architettura a microservizi n-tier in cui ogni componente ha una responsabilità specifica. Questa separazione riduce il rischio che la compromissione di un componente si propaghi all'intero sistema.

L'architettura di RAM-USB è composta da dieci componenti, organizzati in quattro gruppi funzionali:

- **Accesso e registrazione**: Entry-Hub, Security-Switch, Database-Vault.
- **Backup e ripristino**: Storage-Service.
- **Osservabilità**: Mosquitto, Metrics-Collector, Grafana.
- **Rete e certificati**: Network-Manager, Headscale, Certificate-Authority.

4.1.1 Entry-Hub

Entry-Hub è il gateway pubblico del sistema, ovvero il componente esposto a chiunque su internet. Per questo il sistema non gli concede fiducia implicita. Il suo processo gira quindi in un container distroless [14], seguendo il principio di riduzione dell'attack surface raccomandato da NIST SP 800-190 [15]. Senza shell, infatti, la maggior parte delle vulnerabilità critiche note nei container non è sfruttabile [16].

Entry-Hub espone tre endpoint HTTPS: `GET /api/health` e `POST /api/users` sono raggiungibili da chiunque su internet, con certificato firmato dalla CA pubblica Let's Encrypt [17]. `POST /api/login` usa lo stesso certificato, ma ha un listener separato, raggiungibile solo da utenti con accesso alla rete privata, descritta in dettaglio nel Capitolo 7.

Quando riceve una richiesta, Entry-Hub valida il corpo JSON in due fasi: in decodifica, impone un limite di dimensione e rifiuta qualunque campo non atteso nel payload; poi valida i singoli campi email, password e, solo in fase di registrazione, la chiave SSH [18]. Se una di queste verifiche fallisce, risponde HTTP 400 senza specificare quale sia il problema e senza inoltrare la richiesta agli strati più interni del sistema. Se la validazione riesce, inoltra la richiesta a Security-Switch via mutual TLS (mTLS) verificando che il certificato ricevuto appartenga davvero a un Security-Switch. Attende la risposta del Security-Switch e la rilancia al client.

4.1.2 Security-Switch

Security-Switch esiste per evitare che le responsabilità di contattare Database-Vault e Network-Manager siano in mano al punto di ingresso del sistema: riceve le richieste in ingresso solo da Entry-Hub, autenticato via mTLS, le rivalida in modo indipendente e contatta i servizi interni. La logica di validazione, gestione degli errori e logging viene riutilizzata da tutti i servizi principali e vive in librerie interne condivise [19].

Lo scopo della rivalidazione è l'applicazione del principio di defense-in-depth [8].

Se la validazione riesce, inoltra la richiesta a Database-Vault via mTLS.

Nel caso di una richiesta di registrazione, chiede al Network-Manager di creare l'identità del nuovo utente sulla rete mesh e la credenziale (pre-auth key) con cui il suo dispositivo potrà unirvisi.

Nel caso di una richiesta di login, chiede al Network-Manager di concedere un accesso temporaneo di dodici ore per quell'utente verso Storage-Service. Solo quando l'utente avrà ottenuto l'accesso allo Storage-Service, potrà contattarlo e gestire i propri backup.

4.1.3 Database-Vault

Database-Vault è il componente che si occupa di memorizzare le credenziali degli utenti in un database PostgreSQL. È l'unico in grado di accedere al database, ma non ha tuttavia accesso ai file degli utenti, i quali si trovano persistiti sullo Storage-Service. Questa separazione limita l'impatto della compromissione di un singolo componente.

Il diagramma Entità-Relazioni è illustrato in Figura 4.1. Visto che il database deve contenere solo i dati degli utenti e non vi è alcuna relazione significativa da salvare, l'ER è piuttosto semplice.

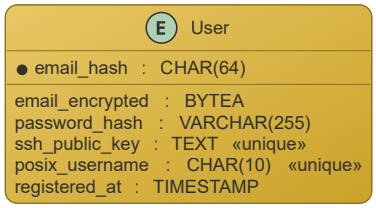


Figura 4.1: Diagramma Entità-Relazioni del record utente in Database-Vault.

L'email è cifrata, recuperabile solo con la master key, mentre la password non è mai recuperabile, poiché viene salvata solo in forma di hash, come digest di Argon2id. Il furto o l'accesso non autorizzato al database non bastano quindi a leggere né l'una né l'altra.

In fase di registrazione di un nuovo utente, in seguito alla rivalidazione dell'input, Database-Vault tenta di salvare i dati in una transazione atomica, rifiutando la richiesta se l'email o la chiave SSH sono già registrate, ma senza specificare quale delle due sia errata. Se il salvataggio riesce, richiede a Storage-Service la creazione dello spazio POSIX dell'utente. In caso di fallimento dello Storage-Service, elimina il record appena creato, altrimenti informa Security-Switch che la registrazione è andata a buon fine.

Al login, ricalcola l'hash della password ricevuta e lo confronta con quello salvato, poi risponde a Security-Switch.

Un'email inesistente e una password sbagliata producono la stessa risposta: in questo modo, chi tenta l'accesso non può usarla per scoprire quali email sono registrate. Il Capitolo 11, tuttavia, spiega come un utente esterno può tentare di capire se un'email è già registrata nel sistema e anche come un amministratore malintenzionato può ottenere le credenziali di accesso di un utente.

4.1.4 Storage-Service

Storage-Service ospita i file di backup degli utenti. Non elabora mai contenuto in chiaro poiché, per design, non è in grado di farlo. Riceve infatti soltanto file deduplicati, compressi e cifrati lato client con Restic [20].

Su richiesta di Database-Vault, crea un utente POSIX dedicato, senza shell interattiva né home directory tradizionale. Ogni utente ha infatti uno spazio scrivibile in una sua sottodirectory dentro un chroot. La struttura del file system è illustrata in Figura 4.2.

```
/storage/ ..... radice di tutti gli utenti registrati
└─ user<xxxxxx>/ ..... proprietario root:root, permessi 0711
   └─ data/ ..... proprietario user<xxxxxx>, permessi 0700
```

Figura 4.2: Struttura del filesystem di Storage-Service, da `creator.go` e `dirmaker.go` [1, 2].

L'accesso allo Storage-Service può avvenire solo via Secure Shell File Transfer Protocol (SFTP) [21], con la chiave pubblica SSH inviata dall'utente in fase di registrazione. Non sono quindi permessi né l'autenticazione con username e password, né altre forme di accesso Secure Shell al di fuori di SFTP.

Ad ogni connessione, un binario `AuthorizedKeysCommand` dedicato interroga Database-Vault per ottenere la chiave pubblica corrente dell'utente. Qualunque errore, in quella chiamata, nega la connessione secondo il principio fail-secure [22].

4.1.5 Network-Manager

Network-Manager è l'amministratore delle regole di rete. Si occupa di definire, modificare ed eliminare le regole che permettono ad un utente della rete privata mesh di raggiungere gli altri membri.

In fase di registrazione, crea un utente Headscale dedicato e la pre-auth key con cui il dispositivo dell'utente si unirà alla rete.

Durante il login, assegna al nodo dell'utente il tag Access Control List (ACL) che abilita l'accesso temporaneo di dodici ore allo Storage-Service. Un processo periodico controlla le scadenze e rimuove il tag dai nodi scaduti.

4.1.6 Certificate-Authority

Certificate-Authority è la CA privata del sistema, basata su `smallstep/certificates` [23].

Emette, rinnova e revoca i certificati mTLS di ogni servizio interno, il che lo rende un componente particolarmente critico del sistema, nonché l'unico raggiungibile da tutti i servizi interni: un malinten-

zionato che riuscisse a comprometterlo, infatti, potrebbe emettere certificati falsificati per qualunque organizzazione.

Ogni servizio ottiene il suo primo certificato contattando la CA sfruttando un token di bootstrap distribuito manualmente da un operatore fuori banda. Una spiegazione dettagliata si trova nella sezione 4.4.

4.1.7 Headscale

Headscale è un'implementazione self-hosted del Tailscale control server [24].

Svolge il ruolo di coordinatore della rete privata mesh: ad ogni tentativo di connessione tra due nodi, fa rispettare le regole di raggiungibilità decise da Network-Manager. Distribuisce inoltre, tramite MagicDNS, i nomi host con cui i nodi si risolvono reciprocamente, permettendo al client di individuare Storage-Service senza conoscerne l'indirizzo IP.

Valida inoltre la pre-auth key di un nuovo dispositivo al momento dell'ingresso nella mesh.

A differenza degli altri componenti interni, fatta eccezione per Entry-Hub, Headscale non risiede nella rete privata, ma è raggiungibile da chiunque su internet. La sezione 7.3 ne approfondisce le ragioni.

4.1.8 Mosquitto

Mosquitto è il broker MQTT su cui ogni servizio, una volta al minuto, pubblica le proprie metriche operative.

Applica una ACL secondo la quale ogni servizio può scrivere sul proprio topic, senza mai potervi leggere, mentre Metrics-Collector può solo leggere l'intero spazio `metrics/#`, senza mai scrivervi.

4.1.9 Metrics-Collector

Metrics-Collector è il componente che riceve, valida e rende persistenti le metriche operative pubblicate da ogni servizio.

Scarta ogni metrica ricevuta il cui campo `service` non corrisponde al topic da cui arriva e persiste le metriche accettate come hypertable in TimescaleDB [25], con ritenzione di trenta giorni e compressione dopo sette giorni.

4.1.10 Grafana

Grafana è la piattaforma di visualizzazione con cui un amministratore consulta lo stato di salute e le prestazioni del sistema [26].

Legge da TimescaleDB direttamente, senza passare per Metrics-Collector. La dashboard di Grafana mostra tempi di risposta, throughput e connessioni attive. Il Capitolo 8 tratta questa pipeline di osservabilità nel dettaglio.

La Figura 4.3 riassume l'architettura appena descritta insieme ai protocolli di comunicazione utilizzati.

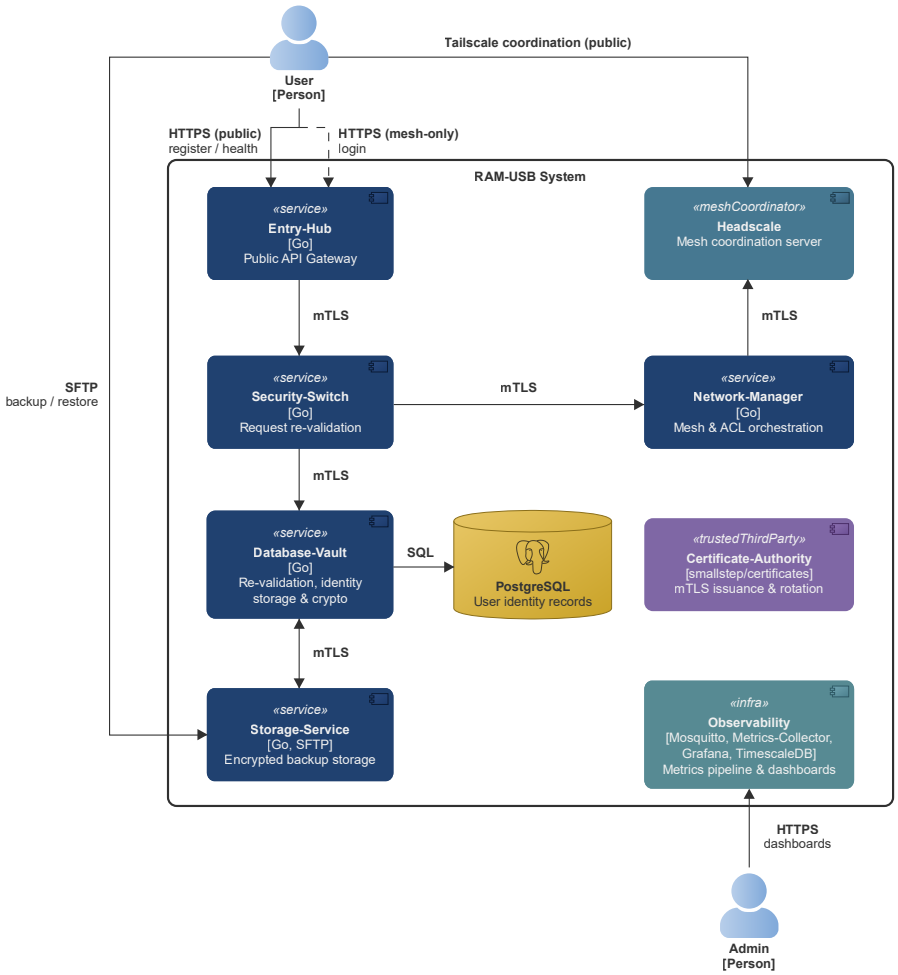


Figura 4.3: Diagramma dei container di RAM-USB.

4.2 Modello di comunicazione inter-servizio

Ogni servizio espone la propria API HTTPS sulla libreria standard di Go, `net/http`. Le richieste e le risposte sono corpi JSON ed ogni endpoint è registrato su un router minimale, `http.ServeMux`. La versione minima di TLS ammessa è la 1.3.

Gli endpoint qui elencati seguono le convenzioni REST [27].

- `GET /api/health`, `POST /api/users` di Entry-Hub,
- `GET /internal/v1/public-key/{posix_username}` di Database-Vault,
- `POST /internal/v1/posix-users` di Storage-Service e
- `POST /internal/v1/mesh-users`, `POST /internal/v1/grants` di Network-Manager

`POST /api/login` è l'unica eccezione, perché un login riuscito restituisce solo la conferma che un'azione è avvenuta ma manca l'identificativo che una vera risorsa `sessions` richiederebbe per essere letta con GET o cancellata con DELETE. Il token ACL di dodici ore che il login concede, infatti, resta interno a Network-Manager e scade autonomamente.

La pubblicazione delle metriche, inoltre, è asincrona: ogni servizio pubblica i propri messaggi su un topic Message Queuing Telemetry Transport (MQTT) dedicato senza alcun bisogno di attendere una risposta.

La struttura del sistema ha un costo non indifferente: ogni registrazione e login attraversa quattro processi distinti, ognuno con il proprio handshake mTLS e la propria rivalidazione. La latenza è quindi ben più alta rispetto ad un sistema monolitico, e non è possibile ridurla saltando uno dei passaggi qualora fosse necessario. Il servizio, inoltre, resta disponibile agli utenti solo finché restano disponibili i sette componenti che partecipano al percorso di una richiesta, mentre la catena di osservabilità può cadere senza che l'utente se ne accorga.

I due costi non sono però dello stesso tipo. La latenza è definitiva, e accettata insieme all'architettura; la disponibilità è invece un problema di distribuzione, e il Capitolo 10 discute fino a che punto l'infrastruttura sottostante sia in grado di attenuarlo.

4.3 Zone di fiducia e confini di sicurezza

La Figura 4.4 mostra le regole di raggiungibilità che Network-Manager applica alla rete mesh.

Le due zone più esterne appartengono all'utente, prima e dopo l'autenticazione. Un utente registrato ma non ancora autenticato, che appartiene quindi alla *Unauthenticated User Zone*, vede e può contattare solo Entry-Hub. Dopo il login, lo stesso nodo entra nella *Authenticated User Zone* e ottiene la raggiungibilità temporanea verso lo Storage-Service.

La Certificate-Authority raggiunge soltanto il broker MQTT, ma è raggiunta da ogni altro componente interno, per permettergli di rinnovare il proprio certificato X.509.

La quarta zona, *Public Zone*, contiene soltanto Headscale, raggiungibile dalla rete pubblica. La sezione 7.3 ne spiega il motivo nel dettaglio. Tuttavia, le chiamate di Network-Manager verso Headscale sono comunque protette dall'autenticazione mTLS e dalla chiave API di Headscale

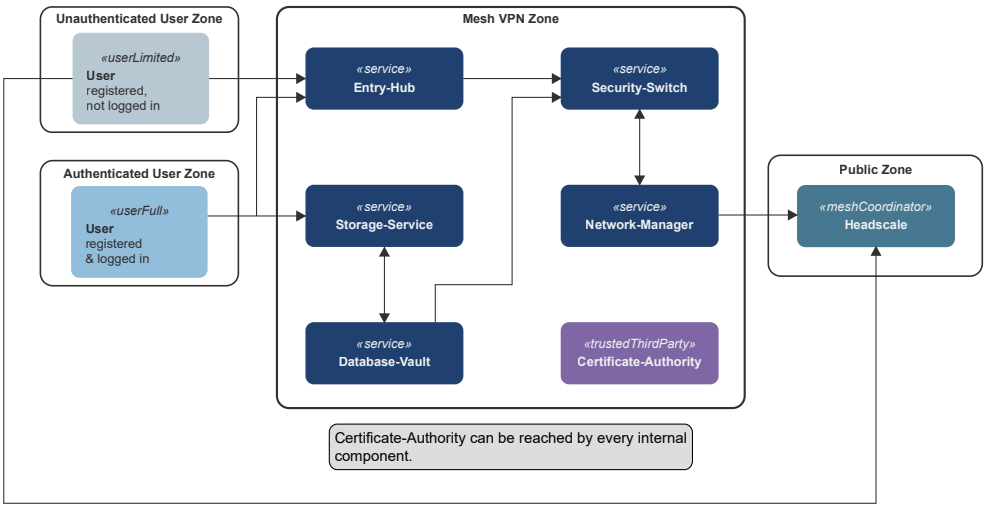


Figura 4.4: Zone di fiducia e raggiungibilità sulla rete mesh.

4.4 PKI e gestione dei certificati

RAM-USB usa due autorità di certificazione, come illustrato in Figura 4.5.

Let's Encrypt, una CA pubblica, firma soltanto il certificato che Entry-Hub presenta sui propri endpoint pubblici. Questo perché il certificato deve essere verificabile da un client esterno che non ha ancora accesso alla rete interna del sistema. Un certificato firmato dalla Certificate-Authority privata di RAM-USB non sarebbe attendibile.

Ogni altra comunicazione usa invece una Certificate-Authority privata, basata su `smallstep/certificates`.

Ogni certificato privato segue lo stesso schema di bootstrap: ogni servizio riceve, fuori banda e una sola volta, un token di bootstrap monouso, e lo presenta alla Certificate-Authority in cambio del proprio certificato iniziale. I rinnovi successivi presentano il certificato mTLS già posseduto. Il token resta un segreto usa-e-getta e non necessita quindi di essere protetto una volta consumato.

Un certificato X.509 valido dimostra solo che la Certificate-Authority ha approvato quel client, non che sia il client atteso per quella connessione. Per questo ogni servizio verifica anche il campo `organization` del certificato.

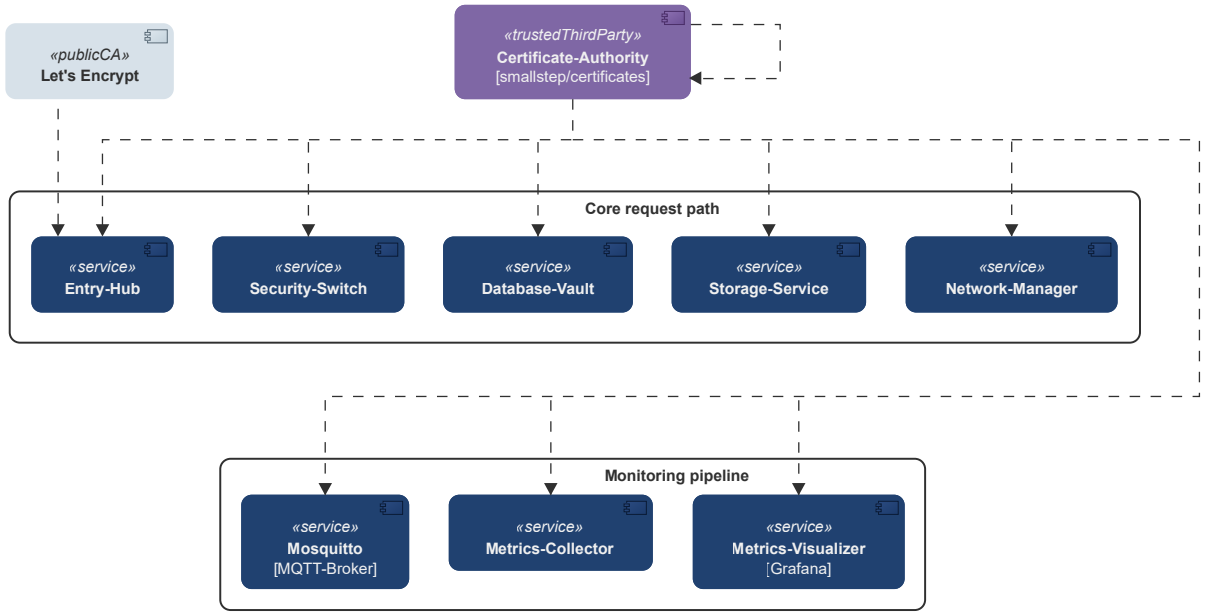


Figura 4.5: Gerarchia delle Certificate Authority e relazioni di firma.

5

Autenticazione e gestione delle identità

5.1 Ciclo di vita della richiesta

Una richiesta di registrazione o di login attraversa tre servizi prima di diventare un record: Entry-Hub, Security-Switch e Database-Vault. Tutti e tre applicano le stesse verifiche sull'input, ognuno indipendentemente dagli altri.

La validazione avviene in due fasi. La prima riguarda il corpo della richiesta: il payload viene letto fino a un limite di 2048 byte e rifiutato se lo supera e il decoder JSON respinge qualunque campo inatteso.

La seconda fase riguarda i singoli campi [18]. L'email deve essere un indirizzo singolo e sintatticamente valido secondo RFC 5322 [28]; la password deve essere lunga tra 8 e 128 caratteri e usare almeno tre categorie tra minuscole, maiuscole, cifre e simboli. In registrazione, inoltre, la chiave pubblica SSH deve essere ben formata.

Se una verifica fallisce, la risposta è HTTP 400 senza indicare quale controllo non sia passato, il log registra il problema senza identificare l'utente, e la richiesta non prosegue verso l'interno. Il messaggio resta generico perché un errore dettagliato aiuterebbe un attaccante più di quanto aiuti un utente in buona fede, che le stesse verifiche le ha già ricevute dal proprio client.

I tre servizi condividono la stessa libreria di validazione, quindi ripetere il controllo non introduce diversità algoritmica, ma garantisce che nessun servizio dia per scontato che il precedente abbia fatto il proprio lavoro. In questo modo, se, per esempio, a Database-Vault dovesse arrivare una richiesta manomessa, proveniente da un servizio compromesso, la validazione verrebbe fatta comunque e farebbe scartare la richiesta.

Gli errori interni vengono infine tradotti in codici HTTP. La Tabella 5.1 riassume i codici che raggiungono l'utente, attribuendo ciascun codice a chi lo emette.

Entry-Hub costruisce soltanto gli errori che nascono da sé stesso: una validazione fallita e una chiamata a Security-Switch andata male. Tutto il resto lo rilancia al client così come gli arriva dall'interno, senza modificarlo. Per questo motivo, un 401 o un 409 raggiungono l'utente pur non essendo mai prodotti da Entry-Hub.

Tabella 5.1: Codici HTTP restituiti al client e loro significato nel sistema.

Codice	**Significato**	**Emesso da**
400	Validazione fallita, senza indicare quale controllo non sia passato	entrambi
401	Credenziali non valide, non specifica se email inesistente o password errata	Security-Switch
403	Rifiuto esplicito di Network-Manager	Security-Switch
409	Email o chiave SSH già registrate, non specifica quale delle due	Security-Switch
500	Errore interno non altrimenti classificato	entrambi
502	Servizio a valle irraggiungibile o con risposta inattesa	entrambi
503	Chiamata a Security-Switch scaduta	Entry-Hub
504	Chiamata a Database-Vault o Network-Manager scaduta	Security-Switch

Tra i codici che i due servizi costruiscono per conto proprio ci sono due differenze: la prima è il 403, che solo Security-Switch può emettere, perché è l'unico dei due a poter ricevere un rifiuto esplicito da un servizio interno. La seconda riguarda le chiamate scadute: Entry-Hub risponde 503, Security-Switch 504.

In tutti i casi, il corpo della risposta è sanificato e il dettaglio dell'errore resta nei log interni.

5.2 Cifratura e hashing delle credenziali

Database-Vault tratta i dati che riceve in tre modi diversi:

- la chiave SSH è pubblica, quindi non necessita di segretezza;
- l'email è insieme una chiave di ricerca e un dato personale da proteggere, ma deve anche essere recuperabile in caso di necessità;
- la password non deve essere recuperabile, nemmeno da chi amministra il database.

L'email viene normalizzata in minuscolo e ridotta al proprio digest SHA-256, che diventa la chiave primaria del record. La normalizzazione garantisce che lo stesso indirizzo scritto con lettere diverse al login produca la stessa chiave della registrazione. Questo hash è deterministico per necessità, perché deve permettere il lookup. Il limite fondamentale di questo approccio, insieme alla sua mitigazione, è discusso nel Capitolo 11.

Accanto al digest che fa da chiave di ricerca, il record conserva una copia dell'email cifrata, così che l'indirizzo resti recuperabile. Da una master key e da un salt casuale di 16 byte, generato per quel record, viene derivata con HKDF-SHA256 [29] una chiave dedicata, con cui l'email è cifrata in AES-256-GCM [30] usando un nonce casuale di 12 byte. Salt, nonce e testo cifrato vengono salvati nel record, mentre la chiave derivata viene azzerata appena il cifrario l'ha consumata. Derivare una chiave per ogni record, invece di cifrare tutto con la master key, riduce la quantità di dati protetti da una singola chiave.

```
func newGCM(masterKey, salt []byte) (cipher.AEAD, error) {
    derivedKey := make([]byte, derivedKeySize)
    kdf := hkdf.New(sha256.New, masterKey, salt, []byte(hkdfInfo))
    if _, err := io.ReadFull(kdf, derivedKey); err != nil {
        return nil, fmt.Errorf("encryption: derive key: %w", err)
    }

    block, err := aes.NewCipher(derivedKey)
    for i := range derivedKey {
        derivedKey[i] = 0
    }
    if err != nil {
        return nil, fmt.Errorf("encryption: new AES cipher: %w", err)
    }

    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return nil, fmt.Errorf("encryption: new GCM: %w", err)
    }

    return gcm, nil
}
```

Codice 5.1: Derivazione della chiave per record in `newGCM`, da `encryption.go` [31].

La password viene invece concatenata a un pepper, letto da una variabile d'ambiente, e passata ad Argon2id [32] insieme a un salt casuale di 16 byte generato sul momento per quel record.

OWASP [33] propone diverse configurazioni equivalenti tra loro, che scambiano memoria e numero di passaggi. RAM-USB parte da quella con più memoria e ne raddoppia i passaggi, pagando quindi a ogni verifica il doppio del lavoro che OWASP considera sufficiente. È un costo che rende impraticabile recuperare le password da un database rubato. Il pepper complica ulteriormente un attacco offline: a differenza del salt, che è salvato nel record e impedisce soltanto che un tentativo valga per più utenti insieme, il pepper nel database non compare. In assenza del pepper, nessun tentativo è computazionalmente praticabile, e una copia del database non è quindi sufficiente a condurre l'attacco.

Il risultato viene salvato come stringa autodescrittiva in formato PHC, la convenzione di codifica nata dalla Password Hashing Competition. Ogni record porta quindi con sé i parametri con cui è stato creato, e la verifica al login li usa al posto delle costanti correnti del servizio. In questo modo, cambiare i parametri per i nuovi utenti non invalida i record già esistenti.

La Tabella 5.2 elenca i campi di quella stringa, nell'ordine in cui compaiono nel record.

Tabella 5.2: Campi della stringa PHC in cui Database-Vault salva l'hash della password.

Campo	Significato
`argon2id`	Variante dell'algoritmo
`v=19`	Versione di Argon2: il byte `0x13` fissato da RFC 9106, stampato in decimale
`m=47104`	Memoria di lavoro in KiB, cioè 46 MiB
`t=2`	Numero di passaggi sulla memoria
`p=1`	Grado di parallelismo
salt	Salt casuale di 16 byte, generato per quel record
digest	Hash di 32 byte prodotto da Argon2id

Master key e pepper risiedono entrambi in variabili d'ambiente, senza una procedura di backup né di rotazione. Il rischio è discusso nel Capitolo 12.

Lo Schema 5.1 riassume le trasformazioni appena descritte, con i simboli seguenti:

- e: l'email ricevuta;
- p: la password ricevuta;
- k_{SSH}: la chiave pubblica SSH;
- π: il pepper;
- MK: la master key;
- s, n, s_p: rispettivamente il salt di HKDF, il nonce di AES-GCM e il salt di Argon2id, generati per quel record;
- $\xleftarrow{R}$: un'estrazione casuale uniforme dall'insieme indicato.

1. $s,\, s_p \xleftarrow{R} \{0,1\}^{128}, \quad n \xleftarrow{R} \{0,1\}^{96}$
2. $k_h = \text{SHA-256}(\text{lower}(e))$
3. $k = \text{HKDF-SHA256}(MK,\, s)$
4. $c = \text{AES-GCM}_k(n,\, e)$
5. $h = \text{Argon2id}(p \,\|\, \pi,\, s_p)$
6. $r = (k_h,\, s,\, n,\, c,\, \text{PHC}(h,\, s_p),\, k_{SSH})$

Schema 5.1: Trasformazioni applicate alle credenziali prima della scrittura del record.

5.2.1 Robustezza della password

Le regole di validazione della sezione 5.1 delimitano lo spazio delle password ammesse, e da quello spazio si ricava, nel caso migliore, quanta incertezza la policy possa garantire a chi tenta di indovinare una password, ossia quante possibilità debba considerare, non sapendo quale sia stata scelta.

Quanto valga quell'incertezza dipende però da come la password è stata scelta. Nel caso migliore, in cui è prodotta da un generatore pseudo-casuale che pesca dall'intero alfabeto ammesso, ogni password ammessa è equiprobabile e l'incertezza è quindi massima. Nel caso peggiore, in cui è costruita da una persona, alcune password sono ben più probabili di altre, ad esempio, `Password1!` viene scelta molto più

spesso di una sequenza casuale della stessa lunghezza. Il valore calcolato qui è quello del caso migliore, cioè un limite superiore.

In questo caso, quindi, il calcolo si riduce a contare i valori distinti che la policy ammette, a partire dalla cardinalità delle quattro categorie:

$$\begin{aligned} n_1 &= 26 && \text{minuscole} \\ n_2 &= 26 && \text{maiuscole} \\ n_3 &= 10 && \text{cifre} \\ n_4 &= 33 && \text{simboli} \\ N &= \textstyle\sum_i n_i = 95 && \text{caratteri disponibili in totale} \end{aligned}$$

Il vincolo di almeno tre categorie esclude le stringhe che ne usano al più due, quindi per contare le password valide conviene contare prima quelle che non lo sono: si parte da tutte le N^L stringhe possibili e si sottraggono quelle che usano una sola categoria e quelle che ne usano esattamente due.

Il primo gruppo è immediato, perché le stringhe scritte con la sola categoria i sono n_i^L. Per il secondo non basta sommare $(n_i + n_j)^L$ su tutte le coppie, perché quel valore conta le stringhe scritte con quei due soli alfabeti e comprende quindi anche quelle di sola i e di sola j: sottraendo n_i^L e n_j^L restano le stringhe che usano davvero entrambe le categorie. Per il principio di inclusione-esclusione, il numero di password valide di lunghezza L vale quindi

$$V(L) = N^L - \sum_{i=1}^{4} n_i^L - \sum_{i<j} \left[(n_i + n_j)^L - n_i^L - n_j^L \right].$$

Un attaccante privo di qualsiasi informazione deve considerare tutti i $V(L)$ candidati ammessi. Quel numero si esprime abitualmente in bit, cioè come esponente di 2: b bit corrispondono a 2^b candidati. Se la password è estratta uniformemente, l'incertezza dell'attaccante vale perciò $\log_2 V(L)$ bit.

Per la lunghezza minima ammessa, $L = 8$, si ottiene $V(8) = 6{,}27 \cdot 10^{15}$, cioè $\log_2 V(8) \approx 52{,}48$ bit.

Il confronto con lo spazio non vincolato mostra quanto pesi ciascuna delle due regole. Senza il vincolo sulle categorie sarebbero ammesse tutte le stringhe di otto caratteri sull'alfabeto, ovvero $N^8 = 95^8 = 6{,}63 \cdot 10^{15}$, cioè 52,56 bit. La suddetta regola quindi riduce i bit di 0,08. Aggiungere un carattere in più, invece, aumenta i bit di entropia di $\log_2 95 \approx 6{,}57$ bit per ogni carattere aggiunto.

Da queste cifre potrebbe sembrare che il vincolo semplifichi il lavoro di un malintenzionato invece di contrastarlo, ed è vero, per quanto in misura trascurabile, nel caso in cui le password siano create da generatori pseudo-casuali. Lo scopo di quella regola però non è aumentare l'entropia, bensì vincolare la scelta di una persona, che tende a fermarsi sulle sole minuscole o su lettere e cifre. Quanto valga in quel ruolo non è ricavabile da questo calcolo.

Se le due regole debbano coesistere, o se convenga sostituire il vincolo sulle categorie con una lunghezza minima maggiore, resta quindi una questione aperta, ripresa tra gli sviluppi futuri nel Capitolo 12.

Quanto quello spazio resista dipende però anche dal costo di ogni tentativo, cioè dai parametri di Argon2id (Tabella 5.2).

5.3 Registrazione

Quando la richiesta raggiunge Database-Vault ed è stata rivalidata, il servizio calcola le tre trasformazioni della sezione precedente e prova a scrivere il record in una singola transazione atomica [34]. Due vincoli di unicità proteggono la tabella, uno sull'hash dell'email e uno sulla chiave pubblica SSH: se uno dei due viene violato, la registrazione è rifiutata con HTTP 409, senza dire quale dei due abbia causato il conflitto. Quel codice però dice a un mittente non autenticato che uno dei due valori inviati era già presente, e chi invia una chiave SSH appena generata sa che l'unico altro candidato è l'email. Il Capitolo 11 analizza questo problema e presenta una possibile mitigazione.

A questo punto il record esiste, ma l'utente non ha ancora uno spazio in cui scrivere. Database-Vault genera allora un nome utente POSIX nella forma `user<xxxxxx>`, dove le sei cifre sono caratteri casuali di un alfabeto base-36, e chiede a Storage-Service di creare l'account corrispondente. Come Storage-Service lo crei e lo isoli è oggetto del Capitolo 6.

Se quella creazione fallisce, il record appena scritto viene eliminato. Se anche il rollback fallisce, il servizio lo segnala con un errore dedicato.

Quando invece entrambi i passi riescono, Database-Vault comunica a Security-Switch che la registrazione è andata a buon fine, e quest'ultimo procede con l'orchestrazione descritta nella sezione conclusiva di questo capitolo.

```
rollbackCtx, cancel := context.WithTimeout(context.WithoutCancel(ctx), rollbackTimeout)
defer cancel()

if delErr := store.DeleteUser(rollbackCtx, input.EmailHash); delErr != nil {
    return Result{
        Outcome: OutcomeFailed,
        Err: fmt.Errorf("%w: posix creation error: %w; delete error: %w", ErrOrphanedRecord,
            err, delErr),
    }
}
```

Codice 5.2: Rollback compensativo dopo un fallimento di Storage-Service, da `registration.go` [35].

La Figura 5.1 riassume l'intera sequenza, dalla richiesta del client fino alla risposta. Gli ultimi due scambi, quelli verso Network-Manager, sono oggetto della sezione conclusiva di questo capitolo.

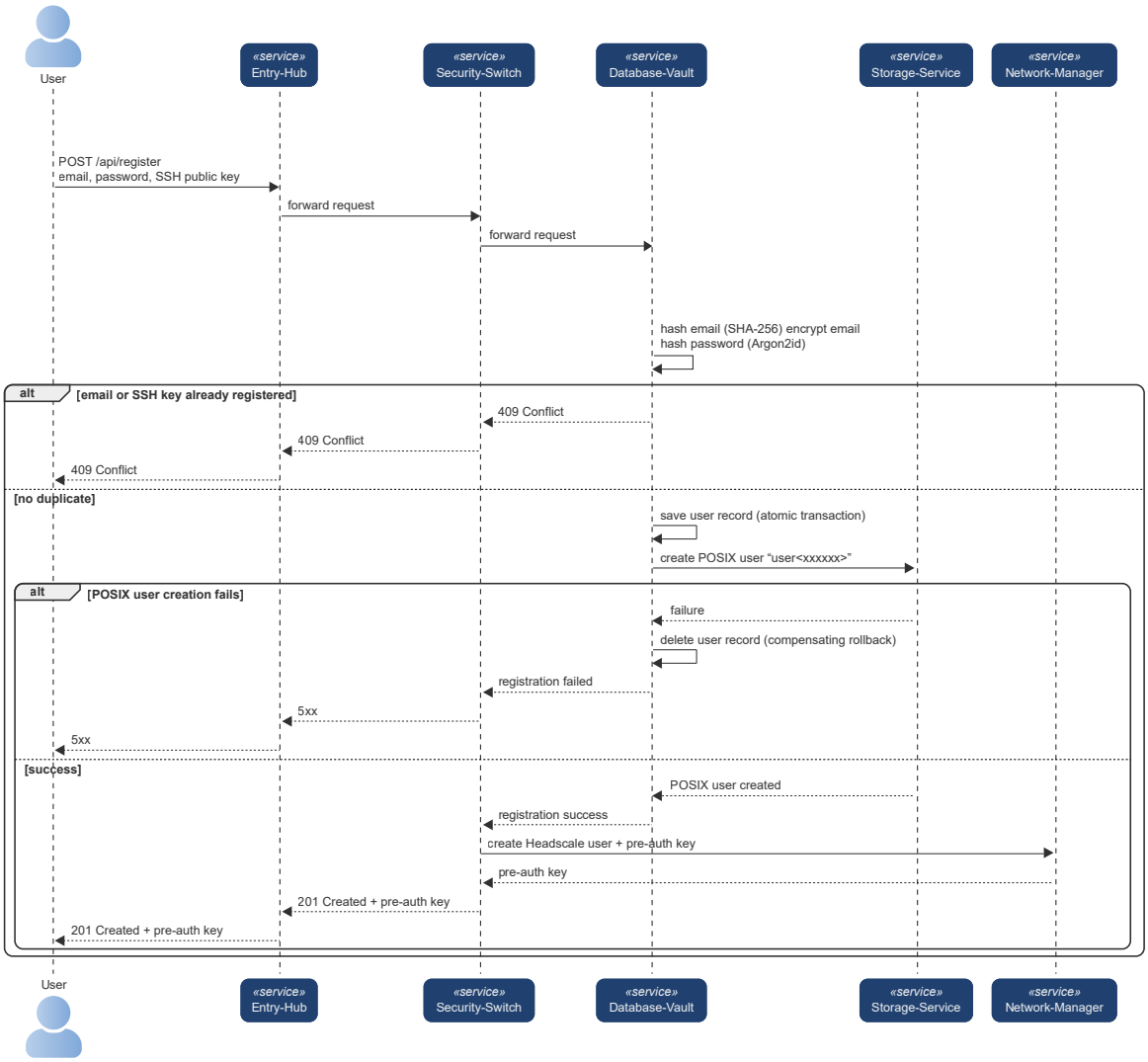


Figura 5.1: Sequenza di registrazione (UC-01).

5.4 Login

Durante il login, Database-Vault ricalcola il digest SHA-256 dell'email ricevuta e con quello cerca il record.

Salt e parametri di costo vengono estratti dalla stringa PHC del record. Argon2id viene ricalcolato con quei valori e con il pepper, e il digest ottenuto è confrontato con quello salvato tramite un confronto a tempo costante, che non lascia quindi dedurre quanti byte iniziali dei due valori coincidano.

Un'email inesistente e una password sbagliata devono produrre la stessa risposta, e non devono essere distinguibili nemmeno nei log. Le due strade convergono quindi su un unico errore sentinella, e il contenuto dell'errore originale viene scartato [36].

Rendere identici la risposta e il log non basta però a rendere indistinguibili i due casi, perché resta una terza differenza osservabile: il tempo. Argon2id, con i parametri della sezione precedente, costa decine di millisecondi, e se venisse eseguito solo quando un record esiste davvero, un attaccante potrebbe

distinguere le due situazioni misurando la latenza della risposta. La soluzione adottata è spendere lo stesso tempo anche quando non c'è nulla da verificare.

```
func VerifyDummy(password, pepper []byte) {
    _, _ = VerifyPassword(password, pepper, dummyHash())
}
```

Codice 5.3: Verifica fittizia usata sui percorsi di rifiuto, da `hash.go` [37].

La funzione ricalcola Argon2id contro un hash fittizio, costruito una sola volta per processo con gli stessi parametri di costo dei record veri, e ne scarta il risultato. Viene chiamata su ogni percorso di rifiuto, compreso quello in cui è il lookup stesso a fallire, così nemmeno un guasto dell'infrastruttura diventa facilmente misurabile dall'esterno.

Ogni incertezza porta allo stesso esito. Se la verifica non può concludersi, per esempio perché l'hash salvato è malformato, la risposta resta 401, secondo il principio fail-secure [22]. Nei log quel caso resta distinguibile, perché a un operatore serve sapere che si tratta di un record corrotto, di un utente distratto o di un vero e proprio attacco.

La Figura 5.2 riassume la sequenza completa di login, compreso il ramo in cui le due cause di rifiuto convergono sulla stessa risposta.

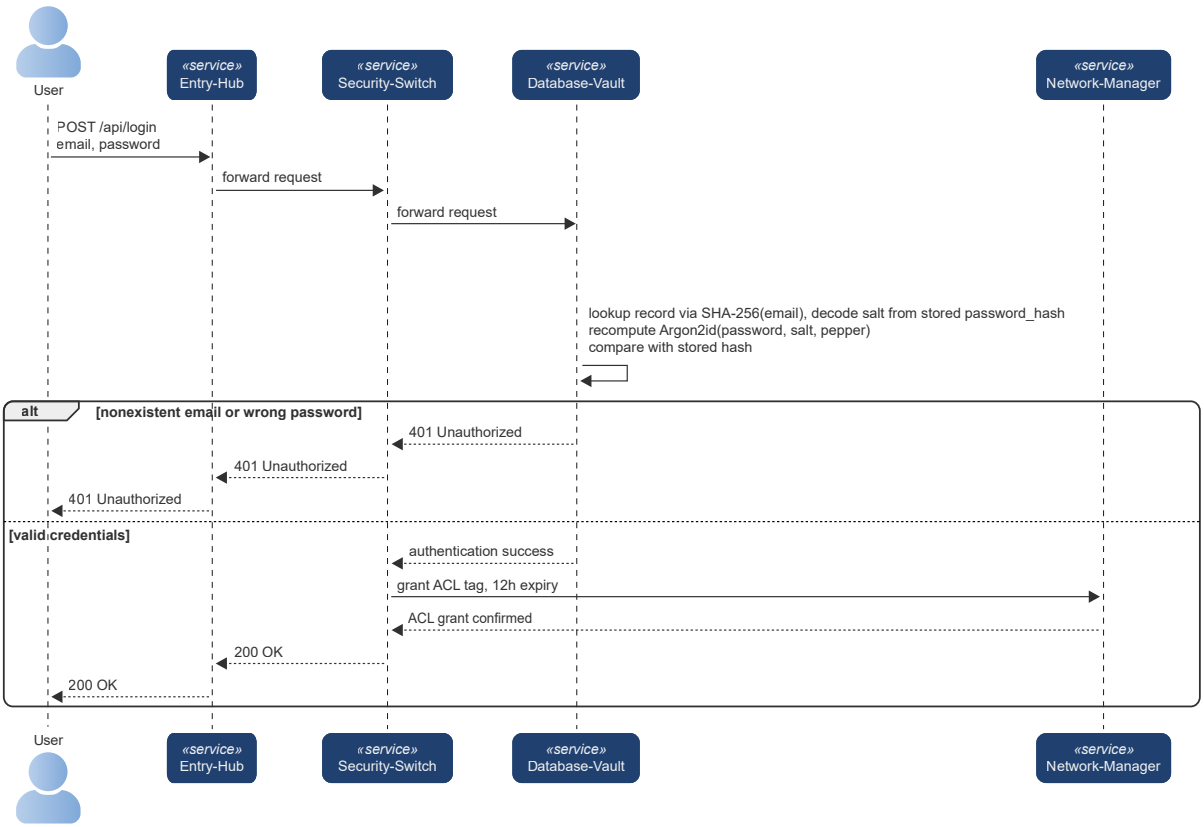


Figura 5.2: Sequenza di autenticazione (UC-02).

5.5 Orchestrazione post-autenticazione

Autenticare un utente non gli dà comunque ancora accesso a niente. L'accesso, in questo sistema, è una proprietà della rete. Security-Switch richiede a Network-Manager la modifica delle regole di rete in due momenti diversi.

Dopo una registrazione riuscita chiede la creazione dell'identità dell'utente sulla rete mesh e di una pre-auth key, che risale la catena fino al client dentro la risposta 201 e che permette al suo dispositivo di entrare nella rete privata.

Dopo un login riuscito chiede invece un accesso temporaneo di dodici ore verso Storage-Service.

Entrambi i passaggi concedono però raggiungibilità, non lettura: la registrazione mette il dispositivo nella rete, il login gli permette di raggiungere Storage-Service, ma ad aprire i backup è la chiave privata SSH, che non lascia mai il dispositivo dell'utente. Conoscere le credenziali dell'utente non è pertanto condizione sufficiente ad accedere ai suoi backup.

Come Network-Manager crei l'identità mesh, applichi il tag ACL e ne gestisca la scadenza è oggetto del Capitolo 7.

6

Storage e backup dei file

6.1 Modello di accesso SFTP-only

Storage-Service conserva i backup degli utenti ed è l'unico componente con cui il client parli direttamente dopo l'autenticazione. Una volta autenticato, infatti, l'utente diventa in grado di raggiungerlo sulla rete privata ed è quindi pronto per gestire i suoi backup.

Prima di trasmetterli però, il client dell'utente, tramite Restic [20], deduplica, comprime e cifra i backup dell'utente. Storage-Service riceve quindi una sequenza di blocchi cifrati che non è in grado di decifrare. Il client, all'inizializzazione del repository Restic, estrae 32 byte da `crypto/rand`, li codifica in base64 e li usa come password del repository [38]. Da quella password Restic ricava la chiave che apre il file in cui custodisce la chiave di cifratura vera, generata a caso alla creazione del repository [39]: è quest'ultima a proteggere i blocchi.

Questo approccio è stato pensato per togliere la possibilità all'utente di scegliere una password debole. Il client salva la password del repository sul dispositivo dell'utente, con gli stessi permessi della chiave privata SSH. La sua segretezza, come la responsabilità di copiare le chiavi in un luogo sicuro, è lasciata all'utente. Nel caso in cui venisse persa anche una sola delle credenziali, l'utente non potrebbe in alcun modo accedere al contenuto dei suoi backup.

Il fatto che i dati degli utenti siano cifrati, però, non significa che debbano poter essere raggiunti da chiunque. L'unico modo per raggiungere i blocchi è infatti una sessione SFTP autenticata, all'interno della rete privata mesh. Su quella sessione, l'unica operazione possibile è trasferire file, e l'unica credenziale accettata è la chiave SSH [40].

Restano fuori gli altri modi di autenticarsi e gli altri usi che SSH consente di norma, come eseguire un comando, aprire una shell, o servirsi della connessione come tramite per raggiungere altre macchine.

Inoltre, solo gli account creati dal servizio possono presentarsi, perché il server accetta esclusivamente nomi utente della forma `user*` e l'accesso root è esplicitamente disabilitato.

La lista predefinita di algoritmi crittografici di OpenSSH [40] è volutamente ampia per garantire compatibilità con client che non supportano gli algoritmi moderni e più robusti, ma qui quella compatibilità non serve, e rischia solo di indebolire il sistema, pertanto, anche gli algoritmi crittografici sono elencati a mano [41].

6.2 Isolamento e chroot dinamico

L'isolamento tra gli utenti si regge su un meccanismo del sistema operativo chiamato chroot, che confina un processo dentro una sottodirectory: da quel momento quella directory è, per lui, la radice del filesystem, e al di fuori di essa non esiste più alcun percorso che il processo possa nominare.

Alla registrazione, Storage-Service prepara per il nuovo utente l'account POSIX che abiterà quello spazio. L'account non ha una home directory tradizionale e la sua shell è `/usr/sbin/nologin`, un programma che si limita a rifiutare l'accesso.

Per ogni utente POSIX il cui nome corrisponde a `user*` vengono applicate anche le regole specificate nel Codice 6.1.

```
Match User user*
    ChrootDirectory /storage/%u # confina la sessione nel sottoalbero dell'utente
    ForceCommand internal-sftp # impone SFTP al posto del comando richiesto dal client
```

Codice 6.1: Le due direttive per utente, da `sshd_config` [41], con commenti aggiunti.

Accanto all'account nasce la struttura di directory illustrata nella Figura 4.2. La radice del confinamento appartiene a root, mentre la sottodirectory `data` appartiene all'utente ed è l'unico posto in cui possa scrivere. La proprietà da parte di root è un requisito del server SSH, che rifiuta di confinare una sessione in una directory scrivibile da chiunque altro.

Il risultato è che un utente collegato vede come radice del proprio filesystem la directory che gli appartiene, e le directory degli altri non sono raggiungibili, né elencabili in alcun modo.

Di norma un server SSH cerca le chiavi che autorizzano una connessione in un file dentro la home dell'utente, dove sono elencate quelle accettate per quell'account. Qui quel file non viene consultato. Al suo posto, a ogni tentativo di connessione, il server esegue un binario dedicato passandogli il nome utente che si sta presentando. Il binario gira con un'identità di sistema che non ha altri compiti sulla macchina e chiede a Database-Vault la chiave pubblica corrente di quell'utente su un canale mTLS.

Qualunque cosa vada storta produce lo stesso esito: il binario termina senza stampare nulla [42]. Il server riceve un elenco vuoto di chiavi e nega la connessione, secondo il principio fail-secure [22]. Non esiste alcun altro luogo al di fuori di Database-Vault in cui siano salvate le chiavi pubbliche SSH, quindi un suo malfunzionamento impedisce agli utenti di avviare le sessioni SFTP.

6.3 Backup e ripristino

Il backup avviene per intero sul dispositivo dell'utente. Il client invoca Restic indicando come repository la sottodirectory scrivibile dentro il proprio spazio confinato, raggiunta via SFTP, e gli impone di autenticarsi con la chiave privata generata alla registrazione [43], che non ha mai lasciato la macchina e che nessun componente del sistema ha mai visto.

Il percorso indicato al repository è già relativo alla radice del confinamento.

Il client risolve Storage-Service tramite MagicDNS sulla rete mesh, e la risoluzione riesce solo finché è attivo il permesso temporaneo concesso dall'ultimo login: scaduto quello, il nodo dell'utente non vede

più il servizio e la connessione non inizia nemmeno. Il Capitolo 7 descrive come quel permesso venga concesso e revocato.

La Figura 6.1 riassume la sequenza, dalla connessione SFTP fino ai dati scritti.

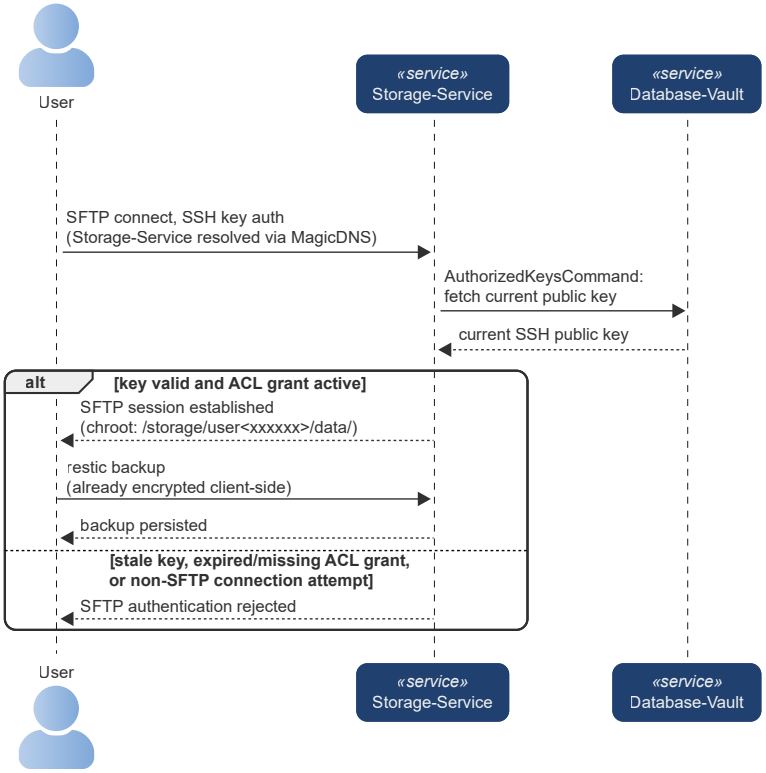


Figura 6.1: Sequenza di backup di un file (UC-03).

Il ripristino percorre la stessa strada del backup; l'unica differenza è il comando impartito a Restic, che scarica i blocchi richiesti e li decifra sul dispositivo dell'utente.

7

Rete e coordinamento della mesh

7.1 Politiche di rete e confini di raggiungibilità

Le zone di fiducia della Figura 4.4 descrivono chi può contattare chi. I confini di raggiungibilità che ne derivano sono resi effettivi da una politica di rete che Network-Manager pubblica su Headscale.

Ogni nodo della rete mesh, sia esso un servizio o il dispositivo di un utente, porta una o più etichette chiamate tag. Le regole di raggiungibilità sono scritte sui tag anziché su indirizzi IP, in modo da sopravvivere al cambiamento di indirizzo di un servizio, dovuto ad esempio al suo riavvio su un nodo diverso.

Network-Manager raccoglie tutte le regole in un unico documento e lo consegna a Headscale una sola volta, all'avvio [44].

Headscale a sua volta lo traduce in un filtro dei pacchetti: a ogni nodo che entra nella rete comunica solo i peer che le regole gli permettono di contattare, insieme ai propri aggiornamenti successivi. In questo modo, un servizio che da regolamento non deve contattarne un altro, non è nemmeno in grado di vederlo nella rete.

Le regole, inoltre, nominano soltanto chi apre la connessione, quindi le risposte viaggiano senza una regola nel verso opposto. In questo modo, ad esempio, Security-Switch può contattare Database-Vault, e quest'ultimo può rispondere alla richiesta, ma non è in grado di avviare una comunicazione con lui.

7.2 Ciclo di vita dei permessi di accesso

Le due regole che riguardano l'utente sono riportate nel Codice 7.1. Il tag `TagStorageAccess` è temporaneo e viene assegnato al login; `TagMeshMember` è permanente e viene assegnato quando il dispositivo entra nella rete.

```
{ // solo un utente autenticato, e Database-Vault, raggiungono Storage-Service
    Action: "accept",
    Src: []string{TagStorageAccess, TagDatabaseVault},
    Dst: []string{dstAny(TagStorageService)},
},
{ // ogni utente registrato raggiunge Entry-Hub
    Action: "accept",
    Src: []string{TagMeshMember},
    Dst: []string{dstAny(TagEntryHub)},
},
```

Codice 7.1: Le due regole che delimitano le zone dell'utente, da `policy.go` [44], con commenti aggiunti.

Alla registrazione, Network-Manager crea per il nuovo utente un'identità su Headscale e una pre-auth key, cioè una credenziale monouso con cui il dispositivo si presenta al coordinatore per entrare nella rete [45]. La chiave vale quindici minuti, e risale la catena dei servizi fino all'utente dentro la risposta alla registrazione.

La chiave nasce già associata al tag di sola appartenenza. Il dispositivo che la usa entra quindi nella rete e ottiene la raggiungibilità verso l'interfaccia di rete privata di Entry-Hub.

Al login Network-Manager cerca il nodo dell'utente e gli aggiunge il secondo tag, poi annota la scadenza a dodici ore da quel momento [46].

La ricerca del nodo dell'utente richiede però che Network-Manager tenga un piccolo database SQLite contenente l'identificativo numerico della pre-auth key legata all'hash dell'email dell'utente, con lo schema riportato nel Codice 7.2.

```
CREATE TABLE IF NOT EXISTS mesh_users (
    email_hash TEXT PRIMARY KEY,
    pre_auth_key_id INTEGER NOT NULL
);
```

Codice 7.2: Schema della tabella di corrispondenza email-pre-auth-key, da `meshusers.go` [47].

Anche le scadenze dei tag sono persistite sul database, per evitare che un riavvio del servizio trasformi un permesso di dodici ore in un permesso perpetuo. Lo schema della tabella che le contiene è riportato nel Codice 7.3.

```
CREATE TABLE IF NOT EXISTS grants (
    email_hash TEXT PRIMARY KEY,
    node_id INTEGER NOT NULL,
    tag TEXT NOT NULL,
    expires_at INTEGER NOT NULL
);
```

Codice 7.3: Schema della tabella dei permessi di accesso, da `store.go` [48].

Un processo periodico, ogni cinque minuti, cerca le scadenze passate e revoca i permessi scaduti [49]. La riga viene cancellata dal database prima di chiamare Headscale per togliere il tag, e la cancellazione è condizionata alla scadenza esatta appena letta. Se nel frattempo un login ha rinnovato il permesso, la

scadenza in tabella non coincide più, la cancellazione non tocca nessuna riga, e lo sweep si ferma senza contattare Headscale. Chiamare prima Headscale e cancellare poi avrebbe invece rischiato di cancellare anche la riga appena riscritta dal login concorrente, togliendo all'utente un permesso appena concesso.

La Figura 7.1 riassume gli stati che il nodo di un utente attraversa.

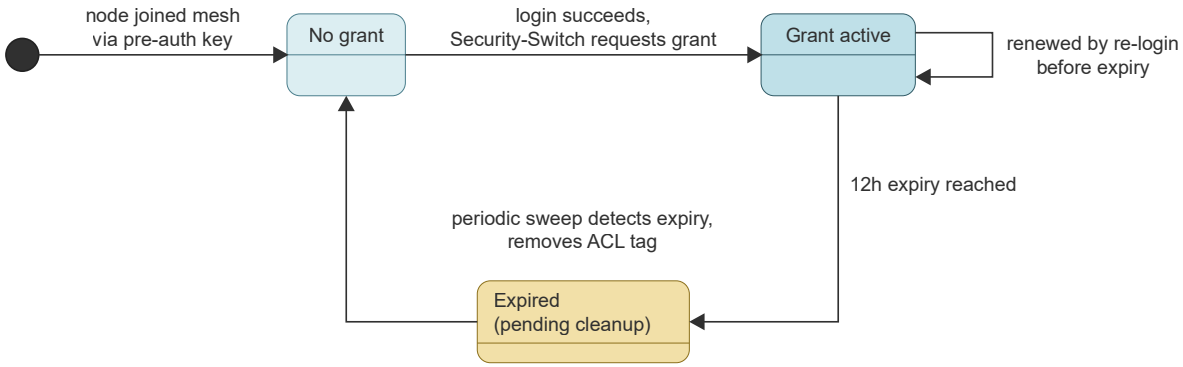


Figura 7.1: Ciclo di vita del permesso di accesso associato al nodo di un utente.

7.3 Il paradosso del coordinatore: Headscale come servizio a sé

La documentazione di Headscale dichiara che non è consigliato eseguire Headscale su una macchina che appartiene alla rete coordinata, perché può creare problemi alla risoluzione dei nomi tramite MagicDNS [50].

Il coordinatore deve perciò restare estraneo alla rete che coordina, e da qui discendono tre conseguenze.

La prima è che non può condividere una macchina virtuale o un container con nessun altro componente, perché tutti gli altri sono membri della mesh.

La seconda è che il traffico con cui Network-Manager crea le chiavi e assegna i tag non può essere protetto dalla collocazione in rete, come invece accade per ogni altra chiamata interna. È l'unica chiamata tra componenti del sistema che attraversa la rete pubblica, e l'unica la cui protezione dipende interamente da mTLS e dalla chiave API di Headscale.

La terza è che il coordinatore deve essere raggiungibile pubblicamente, perché un dispositivo che non è ancora entrato nella rete deve poterlo contattare la prima volta per poter entrare nella rete.

Headscale ha però una limitazione rilevante per RAM-USB: non offre alcun modo di separare il protocollo di coordinamento, che deve essere raggiunto da tutti, dall'API amministrativa, che deve essere raggiunta solo da Network-Manager. La separazione è stata affidata a un reverse proxy, che distingue le due superfici in base al percorso della richiesta [51]. Le richieste di coordinamento passano senza alcun controllo, mentre quelle amministrative sono accettate solo se accompagnate da un certificato emesso dalla Certificate-Authority privata, a cui si aggiunge anche una chiave API che Headscale richiede per conto proprio. Il traffico amministrativo è difeso da due credenziali indipendenti perché è una delle superfici più sensibili.

Il certificato presentato dal proxy è invece emesso da una CA pubblica, per la stessa ragione per cui lo è quello di Entry-Hub.

7.4 CG-NAT e la necessità di un endpoint pubblico dedicato

Il pool centrale degli indirizzi IPv4 non ancora assegnati si è esaurito nel 2011 [52], e per continuare a collegare nuovi clienti molti Internet Service Provider fanno condividere un unico indirizzo IPv4 pubblico a più abbonati. Il router domestico non riceve più un indirizzo pubblico, ma uno del blocco `100.64.0.0/10`, valido soltanto all'interno della rete dell'operatore e non instradabile su internet [53]; è un NAT gestito dall'operatore a tradurlo nell'indirizzo pubblico comune. Questo tipo di NAT prende il nome di Carrier-Grade NAT [54]. Poiché quel NAT non è sotto il controllo del cliente, il cliente non può aprirvi da solo le porte in ingresso, e in generale un server ospitato in casa non è raggiungibile da internet [55]. RAM-USB è progettato per poter essere ospitato anche da utenti domestici, che potrebbero quindi non disporre di un indirizzo IPv4 pubblico né di IPv6. Per questo Headscale ed Entry-Hub risiedono ciascuno su un proprio Virtual Private Server, separato dal resto del sistema. Il Capitolo 10 descrive la collocazione effettiva di ciascun componente.

8

Osservabilità e metriche

8.1 Pubblicazione delle metriche

Le metriche operative possono tipicamente raggiungere chi le osserva in due modi: o è qualcun altro a interrogare periodicamente ogni servizio (*pull*), oppure è ogni servizio a inviarle di propria iniziativa (*push*). RAM-USB adotta il secondo: ogni servizio pubblica le proprie metriche una volta al minuto, sul proprio topic MQTT dedicato verso il broker Mosquitto [56]. La scelta deriva dal fatto che un modello *pull* richiederebbe che Metrics-Collector apra una connessione verso ciascuno dei sei servizi, ampliando le regole di raggiungibilità della mesh discusse nel Capitolo 7. Pubblicando di propria iniziativa, sono invece i servizi ad aprire la connessione verso il broker. Da lì, Metrics-Collector legge le metriche e le persiste in un database.

Le metriche operative non contengono dati degli utenti, ma i metadati del sistema vanno protetti con la stessa cura, per questo le comunicazioni avvengono all'interno della rete privata e protette dal mTLS, certificato dalla CA interna.

Ogni pubblicazione porta come corpo un payload di sei campi, riportato nel Codice 8.1. Le metriche nel payload fanno riferimento al minuto in cui sono state inviate.

```go
type Payload struct {
    Service string `json:"service"`
    Timestamp string `json:"timestamp"`
    RequestCount int64 `json:"request_count"`
    ErrorCount int64 `json:"error_count"`
    AverageResponseTimeMs float64 `json:"average_response_time_ms"`
    ActiveConnections int64 `json:"active_connections"`
}
```

Codice 8.1: Struttura del payload delle metriche, da `payload.go` [57].

Le pubblicazioni usano tutte *QoS 1* (at least once), nella classificazione MQTT [58]: il broker garantisce che il messaggio arrivi almeno una volta, al costo di un possibile duplicato. *QoS 0*, (at most once), è escluso perché RAM-USB non può permettersi di perdere metriche in silenzio; *QoS 2* (exactly once), è escluso perché la garanzia in più che offre (l'assenza di duplicati) qui non serve visto che un duplicato si traduce solo in due punti sovrapposti nello stesso istante, invisibili sul grafico.

8.2 Ricezione e controllo di coerenza

Il messaggio pubblicato deve arrivare a un solo destinatario. Questa proprietà è garantita dall'ACL di Mosquitto che garantisce che ogni servizio possa solo scrivere sul proprio topic, mentre Metrics-Collector può leggere dai topic `metrics/#` senza potervi scrivere.

Quell'ACL, però, non impedisce che un servizio compromesso, pur restando entro il proprio permesso di scrittura legittimo, pubblichi un payload il cui campo `service` dichiarato sia un nome diverso dal proprio. Metrics-Collector rivalida quindi ogni messaggio in modo indipendente, derivando il servizio atteso dal topic di arrivo e confrontandolo con quello dichiarato nel payload. Su qualunque discrepanza scarta il messaggio e lo registra nei log.

8.3 Persistenza delle serie temporali

Metrics-Collector persiste le metriche in TimescaleDB [25], un'estensione di PostgreSQL pensata per un carico fatto quasi solo di scritture ordinate nel tempo.

La tabella `metrics` non ha una chiave primaria, dato che una riga di metriche non ha un'identità naturale. Inoltre, TimescaleDB [25] non ne richiede una. È organizzata in chunk per intervalli di tempo sulla colonna `time`: le due politiche in background del Codice 8.2 agiscono chunk per chunk, in base alla loro età. La prima elimina ogni chunk più vecchio di trenta giorni; la seconda, dopo sette giorni, sposta i chunk in un formato compresso a colonne, raggruppato per servizio e ordinato per tempo decrescente.

```
SELECT create_hypertable('metrics', by_range('time'));

SELECT add_retention_policy('metrics', drop_after => INTERVAL '30 days');

ALTER TABLE metrics SET (
    timescaledb.enable_columnstore,
    timescaledb.segmentby = 'service',
    timescaledb.orderby = 'time DESC'
);
CALL add_columnstore_policy('metrics', after => INTERVAL '7 days');
```

Codice 8.2: Politiche automatiche di ritenzione e compressione della tabella `metrics`, da `000001_create_metrics_table.up.sql` [59].

8.4 Visualizzazione delle metriche

Un amministratore consulta lo stato di salute del sistema, senza che quella consultazione tocchi mai i dati degli utenti visto che per costruzione, le metriche non ne contengono. Grafana [26] legge da TimescaleDB direttamente.

Per contattare TimescaleDB, oltre alla raggiungibilità sulla rete mesh e al certificato mTLS di Grafana, è richiesta anche una password, sullo stesso principio di difesa in profondità già visto per l'API amministrativa di Headscale nel Capitolo 7.

La dashboard mostra tre pannelli, ciascuno una query diretta sulla tabella `metrics`: tempo medio di risposta, throughput e connessioni attive.

La Figura 8.1 riassume l'intera pipeline appena descritta, dalla pubblicazione sul topic MQTT alla dashboard.

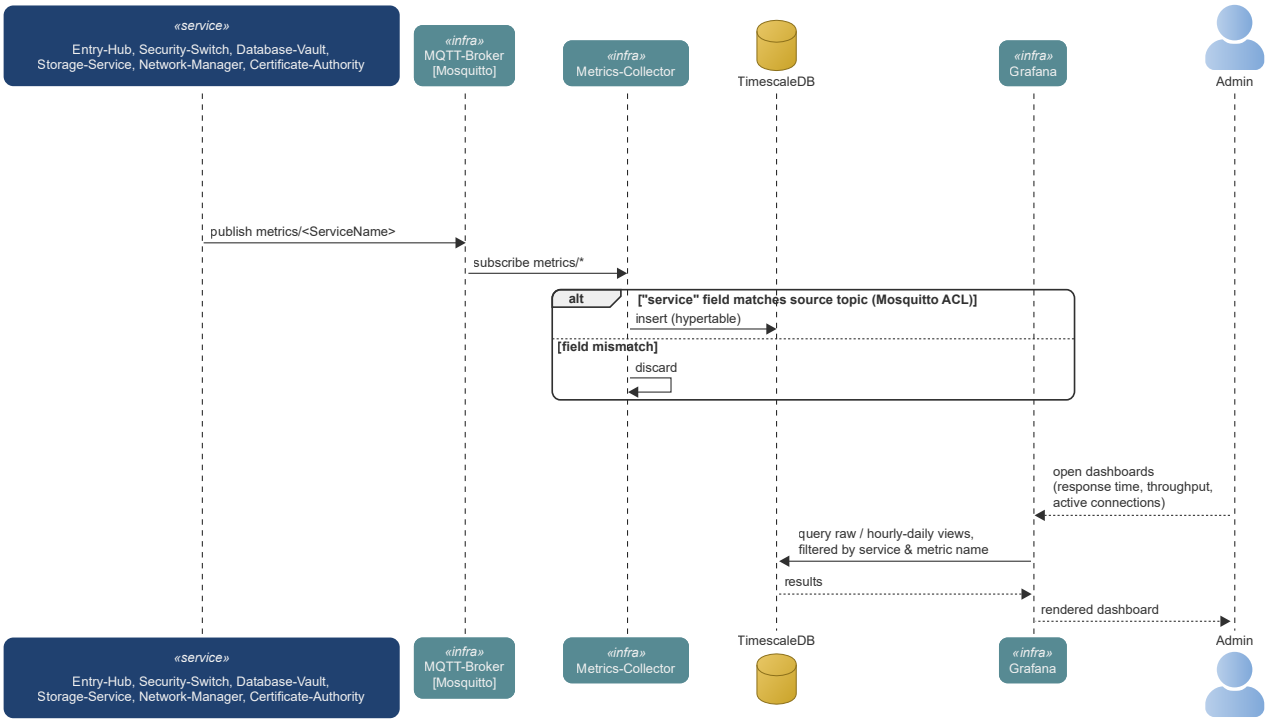


Figura 8.1: Pipeline di osservabilità, dalla pubblicazione delle metriche alla dashboard di Grafana.

9

Testing e validazione

9.1 Il modello V e i quattro livelli di test

Per il testing di RAM-USB è stato adottato il modello a V, un'estensione del modello a cascata in cui ad ogni fase di specifica viene affiancata una fase di verifica corrispondente, scritta di pari passo con la specifica.

Questo approccio è stato adottato in quanto si è scelto di trattare RAM-USB come un sistema safety-critical, in cui un difetto in produzione ha conseguenze serie sugli utilizzatori del sistema. Era pertanto fondamentale che un errore nella specifica venisse rilevato molto prima dell'implementazione e che il testing venisse applicato su vari livelli di specifica.

La Tabella 9.1 riporta la corrispondenza tra la categoria del requisito e il livello di test che lo verifica, con la tecnica usata a quel livello.

Tabella 9.1: Livello di verifica associato a ciascun livello di specifica.

Livello di specifica	**Livello di verifica**
Requisito di componente	Test di unità con il pacchetto `testing` di Go
Caso d'uso	Test di integrazione, bottom-up con driver e stub
Requisito non funzionale	Test di sistema, sull'intero stack
Requisito utente	Lista di accettazione, percorsa a mano sul caso d'uso corrispondente

9.2 Strategia di integrazione bottom-up

L'integrazione parte dal componente più interno e procede verso l'esterno, nell'ordine Database-Vault, Storage-Service, Security-Switch, Network-Manager, Entry-Hub.

La scelta di partire da Database-Vault è dettata dal fatto che questo è contemporaneamente il componente con meno dipendenze uscenti e quello su cui si concentra il maggior numero di requisiti funzionali. Costruirlo per primo, sostituendo con driver i componenti che lo invocano e con uno stub Storage-Service, l'unico che esso invoca, fa sì che il componente più sensibile sia anche quello verificato più a lungo.

9.3 TDD: criteri di ingresso e uscita

Il criterio di ingresso è che per ogni requisito esista un test, e che l'eventuale fallimento di quel test avvenga prima dell'implementazione del requisito da testare.

Un esempio: il requisito DV-F-15 richiede che un'email inesistente e una password sbagliata producano lo stesso esito di login, indistinguibile dall'esterno.

Il test che lo verifica, `TestLogin_NonexistentEmailAndWrongPassword_AreIndistinguishable` [60], chiama `Login` con le due credenziali diverse e pretende che l'esito coincida esattamente.

Un test mai visto fallire, magari scritto dopo il codice e passato al primo colpo, non garantisce di verificare nulla: potrebbe confrontare due valori uguali per costruzione, o non chiamare affatto la funzione che dovrebbe testare. Per questo motivo, il test è scritto prima che il body della funzione `Login` esista davvero, e fallisce, fornendo la prova che il test sa riconoscere un esito sbagliato.

Solo dopo il fallimento del test ha senso scrivere il body completo di `Login`, e solo allora il passaggio da test fallito a test passato diventa un'informazione utile.

Scrivere i test prima aiuta anche a disambiguare e a chiarire i requisiti. Se per un requisito non si riesce a scrivere un test, infatti, significa che il requisito è poco chiaro o ambiguo e di conseguenza, deve essere riformulato meglio.

I criteri di uscita sono tre, e dipendono dal codice da testare. Il primo riguarda solo la libreria di validazione, che richiede test più approfonditi perché considerata particolarmente sensibile.

- nella libreria di validazione condivisa da Entry-Hub, Security-Switch e Database-Vault (sezione 5.1), ogni ramo di ogni `if` o `switch` che decide se accettare o rifiutare un campo deve essere attraversato da almeno un caso di test;
- nessun test già passato sulla catena di componenti già integrata può fallire;
- una modifica non riesegue solo il test scritto apposta per lei, ma l'intera suite di test.

9.4 Copertura e lacune del piano di test

`go test -cover` misura, per ogni pacchetto della codebase, la percentuale di istruzioni eseguite almeno una volta: `pkg/validation` ad esempio, arriva al 98%, perché lì vive la logica applicativa, mentre i pacchetti `cmd`, che avviano ciascun servizio, restano al 10–20%, perché verificarli è compito dei test di integrazione e di sistema.

Il criterio di uscita citato sopra, per `pkg/validation`, è una misura più severa e si aggiunge a questa: non basta eseguire quasi tutte le istruzioni, ogni singolo ramo di ogni `if` o `switch` deve essere stato preso da almeno un test.

Il piano di test, però, non è esente da limitazioni: Non esiste alcun test previsto né per la protezione dei backup dalla manomissione, né per la rotazione della master key, attualmente fuori perimetro, ma fondamentali in un progetto di questo tipo. I test di carico e di stress, inoltre, sono previsti, ma non ancora progettati. Non si hanno quindi benchmark affidabili del comportamento del sistema. Ogni test end-to-end sull'host ha riguardato un solo utente con un solo piccolo file nel backup, quindi né la concorrenza né l'isolamento reciproco fra due utenti sono stati messi alla prova.

9.5 Verifica automatica pre-commit

Oltre ai criteri di ingresso e di uscita, una catena di verifica automatica viene eseguita prima di ogni commit [6]. `golangci-lint` [61] è un esecutore che aggrega più linter in un solo passaggio; quello configurato per RAM-USB ne abilita quindici in tutto.

Tra questi, `gosec` [62] cerca pattern di sicurezza noti nel codice, ed è presente fin dalle prime fasi di implementazione.

Una seconda estensione ha introdotto quattro linter mirati a risorse mai rilasciate e a errori gestiti male:

- `bodyclose` [63] segnala una risposta HTTP il cui corpo non viene mai chiuso, che porta quindi ad una perdita di risorse;
- `sqlclosecheck` [64] fa lo stesso per righe e statement SQL rimasti aperti;
- `contextcheck` [65] individua una funzione che non propaga il `context.Context` ricevuto dal chiamante;
- `errorlint` [66] verifica che gli errori vengano confrontati nel modo corretto.

A questi si affianca il rilevatore di race-condition di Go, invocato con `go test -race`.

Completano la catena `govulncheck` [67], che confronta le dipendenze con un database di vulnerabilità note ma segnala un problema solo se il codice raggiunge davvero la funzione vulnerabile, e due scanner pensati per sovrapporsi tra loro: `gitleaks` [68], che cerca segreti già committati nella storia del repository, e `trivy` [69], invocato in modalità di scansione del filesystem (`trivy fs .`), che verifica sia le dipendenze sia i segreti in modo indipendente dai due strumenti precedenti.

La ridondanza degli ultimi due è l'applicazione ai test del concetto di difesa in profondità.

9.6 Risultati: verifiche completate

La campagna più significativa è stata eseguita su hardware reale, un host Proxmox con i componenti distribuiti secondo la collocazione descritta nel Capitolo 10.

I quattro casi d'uso sono passati: registrazione e login sono riusciti, il backup si è concluso e il ripristino ha restituito un file identico all'originale.

La Tabella 9.2 riporta le proprietà controllate sul sistema in esecuzione.

Tabella 9.2: Proprietà verificate sul sistema in esecuzione.

Proprietà	**Riscontro**
Parametri di hashing	La stringa salvata nel record riporta `argon2id`, versione 19, memoria 47104 KiB, due passaggi, parallelismo uno
Isolamento su filesystem	Radice del confinamento di proprietà di root con permessi 711, sottodirectory dell'utente 700
Identità sulla mesh	Ogni nodo risulta di proprietà dell'utente sintetico `tagged-devices`, mai dell'account creato per la persona
Durata del permesso	Scadenza registrata a dodici ore esatte dall'istante del login
Confini di raggiungibilità	Nessun nodo elenca fra i propri peer un componente che la politica gli vieta di contattare
Sopravvivenza al riavvio	Un servizio fermato e riavviato torna in servizio riusando l'identità su disco, senza ripresentare il token monouso già consumato

Durante l'esecuzione del piano di test completo, sono emersi diversi difetti non intercettati dai test di unità che li precedevano [70].

Due riguardavano il client e impedivano del tutto il backup. Il primo era la porta SSH: il client si collegava su quella predefinita, mentre il server ascolta su un'altra. Il secondo era l'assenza di qualunque politica sulla chiave host: `ssh`, invocato da Restic senza terminale, non aveva un `known_hosts` da controllare né modo di chiedere conferma, e rifiutava la connessione anziché chiederla, quindi il primo backup su ogni macchina nuova falliva sempre. Un terzo riguardava la configurazione, perché il client dispone di un solo indirizzo per un servizio che ne espone due su listener diversi. Un quarto era una stringa d'uso che documentava una sintassi che il parsing degli argomenti non può accettare.

Un quinto, più grave: l'identità mTLS non veniva conservata fra un avvio e il successivo, quindi nessun servizio sopravviveva al riavvio del proprio container, perché ritentava il bootstrap con un token monouso ormai speso.

Altri piccoli errori di logica e implementazione, troppo numerosi per essere riportati in questa tesi, sono stati rilevati e corretti grazie ai test.

10

Distribuzione

10.1 Containerizzazione e supervisione s6-overlay

Alcuni servizi di RAM-USB devono far girare più processi indipendenti nello stesso container e necessitano quindi di un supervisore che li avvii nell'ordine giusto e li riavvii quando cadono. Quello usato da RAM-USB è `s6-overlay` [71]. Ogni processo supervisionato è una directory sotto `s6-rc.d/`, dichiarata di tipo `longrun`; le sue dipendenze sono file dentro una sottodirectory `dependencies.d/`, uno per ogni processo che deve essere avviato: se, ad esempio, il processo A deve aspettare che il processo B sia pronto, `A/dependencies.d/` contiene un file vuoto chiamato `B`. L'ordine di avvio nasce quindi da un grafo dichiarato tra directory. Storage-Service è il caso più complesso: lo stesso container Docker è condiviso da 4 processi [72]: `tailscaled`, il client mesh reale; `sshd`, il server SFTP (Capitolo 6); `identity-provisioner`, che mantiene aggiornato il certificato X.509 per il mTLS; e il binario Go dello stesso Storage-Service, che espone l'API HTTP interna. Ogni processo `longrun` ha anche un file `run` e un file `finish`. Il primo è lo script che lancia il binario vero e proprio [72]. Se quel processo termina, il supervisore lo rilancia da capo; il secondo è opzionale e serve a ritardare il rilancio di qualche secondo, in modo da evitare che un errore di avvio irreversibile diventi un ciclo di crash continuo.

Non tutti i servizi, però, hanno bisogno di questa complessità. Entry-Hub, ad esempio, non supervisiona alcun processo secondario: la sua appartenenza alla mesh vive in un sidecar Docker separato, `entry-hub-mesh`, che condivide il suo network namespace. Il paragrafo seguente spiega in che modo questa differenza è legata alla scelta dell'immagine di base di ciascun servizio.

10.2 Perché non tutti i container sono distroless

Un'immagine distroless [14] non ha shell né gestore di pacchetti, riducendo così numerose vulnerabilità critiche note nei container [16]. Un supervisore come `s6-overlay`, però, ha bisogno di una shell per i propri script di avvio. Qualunque servizio debba far girare più di un solo binario Go non può quindi essere distroless. Di conseguenza, l'unico servizio distroless è Entry-Hub [73], mentre gli altri sei usano `debian:bookworm-slim` o `alpine`, come riportato in Tabella 10.1.

Tabella 10.1: Immagine di base per servizio e processi che condividono lo stesso container.

Servizio	**Immagine di base**	**Condivide il container con**
Entry-Hub	distroless	nessuno (mesh in un sidecar separato)
Security-Switch	debian:bookworm-slim	`tailscaled`
Network-Manager	debian:bookworm-slim	`tailscaled`
Storage-Service	debian:bookworm-slim	`tailscaled`, `identity-provisioner`, `sshd`
Database-Vault	`postgres:17` (Debian)	`tailscaled`, `postgres`
Metrics-Collector	`timescaledb` (Alpine)	`tailscaled`, `timescaledb`
Certificate-Authority	`step-ca` (Alpine)	`step-ca` (mesh in un sidecar separato)

Estendere distroless a più servizi o scegliere delle distribuzioni diverse da quelle elencate in Tabella 10.1 è un problema non ancora affrontato.

10.3 Docker locale vs Proxmox VE: due livelli di isolamento

Sarebbe naturale pensare a Docker e a Proxmox VE [74] come due alternative per la stessa esigenza di isolamento. In realtà operano a due livelli diversi. Docker è un motore di containerizzazione: isola un processo a livello di sistema operativo, condividendo il kernel con l'host, così che ogni container veda un proprio filesystem, una propria rete e un proprio albero di processi senza il costo di un secondo kernel [75]. Garantisce così portabilità su ogni sistema operativo e isolamento per-servizio, identici in sviluppo e in produzione.

Proxmox aggiunge invece, sopra ciascun container Docker, il confine di isolamento di una macchina virtuale KVM, con il proprio kernel, o, in alternativa, quello di un container di sistema LXC, più leggero ma che condivide il kernel con l'host. Ogni servizio gira quindi dentro il proprio container Docker, ma quel container Docker gira a sua volta dentro la sua macchina virtuale KVM o dentro il suo container LXC. La scelta di Proxmox porta anche un altro vantaggio: apre la strada alla migrazione a caldo delle KVM e, con più nodi, all'alta disponibilità.

Storage-Service, Database-Vault e Network-Manager sono assegnati a guest KVM, mentre i restanti servizi sono assegnati a container LXC non privilegiati, in cui quindi l'utente root del container è mappato su un utente non root dell'host. Entry-Hub e Headscale restano fuori da Proxmox del tutto, su un VPS pubblico dedicato ciascuno, per i motivi spiegati nel Capitolo 7.

La scelta tra KVM e LXC dipende, per quasi tutti i servizi, dal livello di isolamento che questi richiedono. I container LXC condividono il kernel con il sistema host, perciò non sono completamente isolati, ma necessitano di pochissime risorse, quindi l'host può sopportarne diversi contemporaneamente. Per questo motivo sono la scelta di default. I servizi che tengono i dati degli utenti, la gestione delle politiche di rete e i backup, necessitano di maggiore isolamento, perché la loro compromissione potrebbe causare dei danni irrimediabili. Storage-Service ha inoltre bisogno di eseguire `setuid` e `chroot` per isolare gli utenti POSIX (sezione 6.2): farlo dentro un container LXC, già esso stesso un namespace, aggiungerebbe un secondo livello di mappatura degli UID. Una KVM evita di sovrapporre due meccanismi di isolamento che non compongono in modo pulito [76].

10.4 Topologia distribuita

Il repository contiene undici file Compose separati sotto `deployments/compose/` [77]. Dieci corrispondono ai componenti interni (Capitolo 4); l'undicesimo, `tailscale-test.yml`, è di supporto ai test. La containerizzazione del client è invece affidata agli sviluppi futuri. Il file di ciascun componente resta indipendente dagli altri perché ogni servizio è pensato per poter essere avviato, fermato o spostato su un nodo Proxmox diverso, in una KVM o LXC differente.

La Figura 10.1 riassume la topologia di produzione.

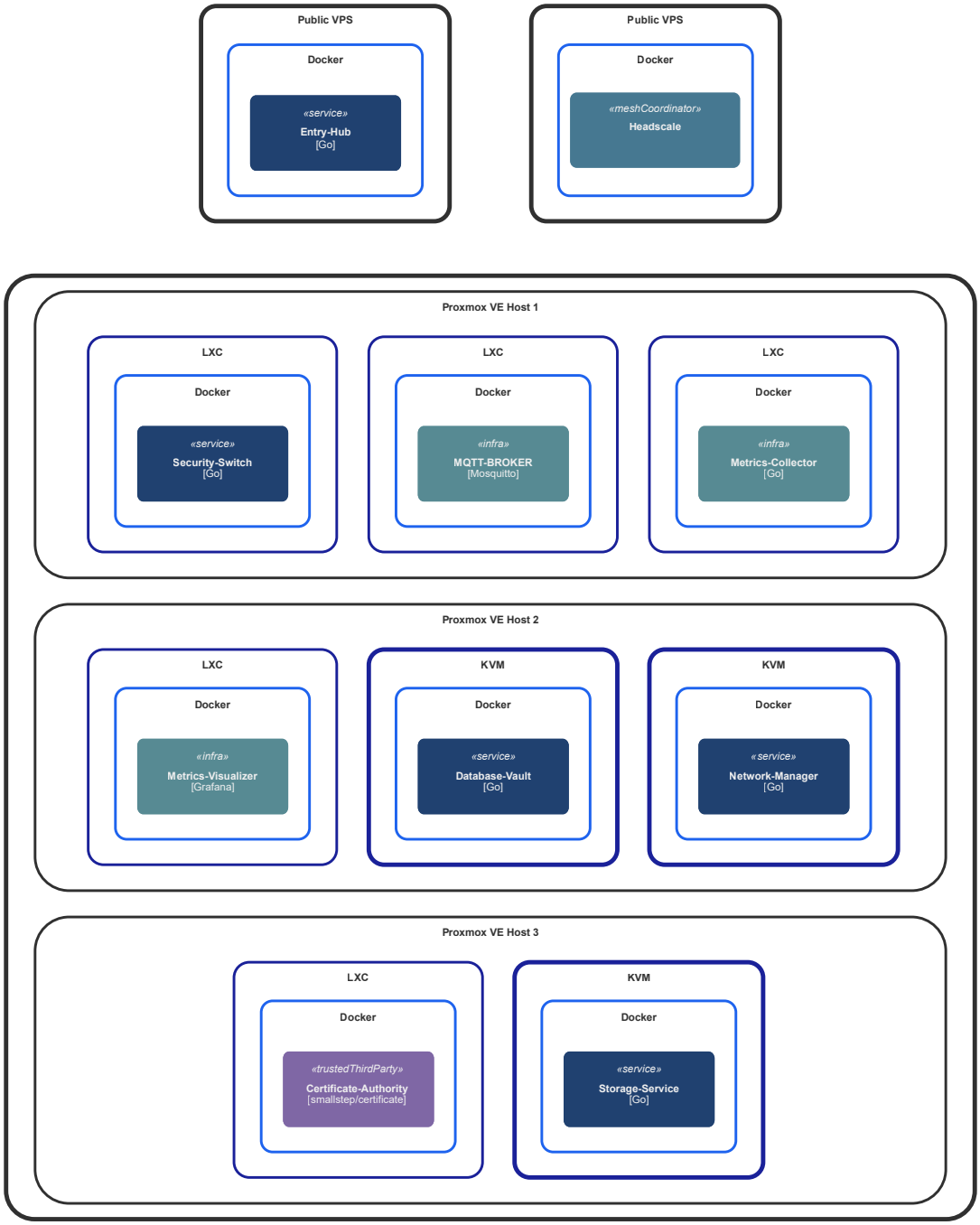


Figura 10.1: Topologia di produzione target.

10.5 Alta disponibilità e resilienza: stato attuale e limiti

Il servizio resta disponibile agli utenti solo finché restano disponibili i componenti che partecipano al percorso di una richiesta.

Il numero di nodi fisici disponibili determina quanta resilienza e tolleranza ai guasti sia raggiungibile da RAM-USB: ogni nodo aggiunto sblocca una proprietà in più.

Con un solo nodo fisico, la supervisione di `s6-overlay` riavvia un processo caduto dentro un container Docker (sezione 10.1), ma non copre l'indisponibilità del nodo fisico su cui gira il servizio. Inoltre si può al massimo ottenere la ridondanza locale dei dischi, per esempio con un RAID-10.

Un secondo nodo sblocca la migrazione senza downtime delle KVM da un nodo all'altro, per bilanciare il carico o liberarne uno per manutenzione [78]. I container LXC, invece, non sono migrabili a caldo, solo riavviabili altrove [79].

Con almeno tre nodi e lo storage replicato con Ceph, il sistema diventa un cluster iperconvergente, in grado di offrire anche il recupero automatico dopo la caduta di un nodo fisico, oltre allo spostamento pianificato di uno sano.

I nodi, però, per poter cooperare devono trovarsi sulla stessa rete a bassa latenza poiché lo stack del cluster Proxmox richiede una latenza inferiore ai 5 ms tra tutti i nodi [80] per operare in modo stabile.

Anche con la migliore fibra disponibile, due nodi lontani tra loro non possono raggiungere una latenza così bassa. Estendere il backend su Proxmox a più sedi richiederebbe un meccanismo diverso, non ancora previsto. Iperconvergenza e geo-distribuzione, allo stato attuale, si escludono a vicenda. Ottenerle entrambe contemporaneamente resta un problema aperto fuori dal perimetro di RAM-USB.

11

Analisi critica delle proprietà di sicurezza

11.1 Scenari di attacco e mitigazioni

I capitoli precedenti descrivono le difese di RAM-USB dal punto di vista di chi le costruisce. Questa sezione le rilegge dal punto di vista di chi tenta di comprometterle.

11.1.1 Enumerazione degli indirizzi email in registrazione

`POST /api/users` è un endpoint pubblico che risponde 201 se l'email inviata è disponibile e 409 se è già registrata, senza specificare se il conflitto viene dall'email o dalla chiave SSH. Chi invia però una password validata dal client e una chiave SSH generata al momento, quindi certamente unica, ha già escluso che il conflitto sia sulla chiave SSH.

Lo Schema 11.1 riassume il procedimento, con i simboli seguenti:

- e: l'indirizzo email candidato;
- p: una password validata dal client;
- k: una chiave pubblica SSH generata dall'attaccante;
- r: il codice di stato HTTP restituito da `POST /api/users`.

1. $k \leftarrow \text{KeyGen}()$
2. $r \leftarrow \text{POST } \texttt{/api/users}(e,\ p,\ k)$
3. $r = 409 \implies e$ già registrata

Schema 11.1: Verifica se un indirizzo email è già registrato.

Il login, invece, nasconde molto meglio questa informazione, restituendo lo stesso 401 sia per un'email inesistente sia per una password sbagliata, con lo stesso log e con lo stesso tempo di risposta (sezione 5.4). Quella protezione non è mai stata estesa alla registrazione.

Esiste anche un effetto collaterale ben più grave dell'enumerazione sull'endpoint di registrazione, causato dal fatto che il sistema non verifica che chi ha inviato le credenziali per la registrazione le possieda davvero.

Un malintenzionato che si registra con un'email che non gli appartiene, impedisce per sempre la registrazione al suo legittimo proprietario, obbligandolo ad usare un indirizzo email diverso.

Entrambi i problemi sarebbero risolvibili rispondendo alle richieste di registrazione in modo identico in tutti i casi, per esempio sempre con un 202, mentre l'esito reale viaggerebbe fuori banda, in un'email di conferma inviata all'indirizzo indicato.

Questo però richiede di ripensare a come restituire all'utente la pre-auth key che oggi viaggia dentro la risposta 201 della registrazione.

11.1.2 Accesso amministrativo alle credenziali

Email e password attraversano, cifrate in transito, Entry-Hub, Security-Switch e Database-Vault. Chi controlla anche uno solo di questi processi può leggerle nel punto in cui sono in chiaro in memoria.

A riposo, però, la password non è mai recuperabile, nemmeno da un amministratore con pieno accesso al database. L'email invece è recuperabile da un amministratore che possiede la master key.

Il problema delle credenziali catturate in transito si potrebbe mitigare cifrando end-to-end le credenziali tra il client e il Database-Vault, ma in questo caso, il problema non verrebbe comunque completamente risolto e si introdurrebbe aggiuntiva complessità al sistema. L'unica vera soluzione è l'utilizzo di un sistema di autenticazione zero-knowledge.

11.1.3 Verifica offline di un indirizzo a partire da una copia del database

Un attaccante che possiede una copia rubata del database può immediatamente verificare se un indirizzo email sospettato è presente o meno nel database. Gli basta calcolare l'hash SHA-256 dell'indirizzo candidato e confrontarlo con gli hash salvati nel database.

Estrarre una email qualunque dal database, però, richiede uno sforzo ben diverso. L'attaccante non può decifrare l'email senza la master key ed è quindi costretto ad attaccare il suo hash SHA-256. Potrebbe, tuttavia, identificare alcune email se queste fossero in dizionari pubblici, utilizzando un attacco a dizionario, altrimenti l'unica via percorribile rimane un attacco brute force.

Lo Schema 11.2 formalizza il procedimento, con i simboli seguenti:

- D: l'insieme dei digest presenti nel database rubato;
- W: un dizionario di indirizzi plausibili;
- e: un indirizzo email estratto da W.

1. $\forall e \in W$:
2. $k_h \leftarrow \text{SHA-256}(\text{lower}(e))$
3. $k_h \in D \implies e$ già registrato

Schema 11.2: Ricerca di indirizzi registrati a partire dal database rubato, per dizionario o forza bruta.

Entrambe queste vulnerabilità sono mitigabili sostituendo il digest semplice con un HMAC-SHA256 con chiave il pepper, generato con un'entropia sufficiente e differente da quello per l'hashing delle password. Un utente che ruba il database non ottiene automaticamente anche il pepper, e non è quindi in grado di ottenere le email in tempi ragionevoli.

Il furto delle credenziali degli utenti, seppur molto grave, non compromette i backup e non impedisce all'utente di accedervi. Per quello serve la chiave privata SSH, che è tenuta localmente dall'utente. Non è nemmeno sufficiente ad eliminare o modificare l'account dell'utente perché il sistema non garantisce il diritto all'oblio agli utenti.

11.2 Isolamento dei fallimenti

Questa sezione analizza quali tra i Requisiti Utente restano garantiti anche nel caso peggiore: la compromissione totale di un servizio. Analizza, inoltre, come risolvere la compromissione dove possibile, a patto che questa venga rilevata.

Tabella 11.1: Requisiti Utente verificati in questa sezione contro la compromissione totale di un servizio.

Requisito	**Descrizione**
RU-04	Poter accedere al proprio backup solo dopo essersi autenticato.
RU-05	Avere la garanzia che nessuno, amministratori inclusi, possa leggere i propri backup.
RU-06	Avere la garanzia che nessuno, amministratori inclusi, possa leggere le proprie credenziali di accesso.
RU-08	Avere la certezza che i propri dati siano isolati da quelli degli altri utenti.

Le restanti proprietà sono fuori dalla verifica: RU-01, RU-02, RU-03, RU-07 e RU-10 sono requisiti funzionali o di disponibilità che un servizio compromesso nega facilmente, e non richiedono quindi una verifica caso per caso; RU-09 resta fuori perimetro come dichiarato al Capitolo 3.

11.2.1 Entry-Hub

Un attaccante che ottenga il controllo di Entry-Hub può leggere, bloccare o manomettere ogni richiesta di registrazione e login. Può anche abusare della sua raggiungibilità verso Security-Switch, Certificate-Authority e il broker MQTT.

La manomissione però non intacca RU-05 perché la password del repository Restic nasce sul dispositivo dell'utente e non attraversa mai Entry-Hub, prima, durante o dopo una manomissione (Capitolo 6). Non intacca neanche RU-04, perché un utente già autenticato raggiunge Storage-Service direttamente sulla mesh, senza mai passare da Entry-Hub (Capitolo 4). Può tuttavia bloccare o alterare le nuove richieste di registrazione e login.

La compromissione, se rilevata, può essere facilmente contenuta. Network-Manager può revocare l'appartenenza di un nodo alla mesh, tagliando fuori dalla rete l'istanza di Entry-Hub compromessa, e mentre si indaga sulla causa della compromissione, una nuova istanza di Entry-Hub può essere avviata in una KVM su Proxmox, idealmente su un nodo diverso (sezione 10.3). In questo modo, la registrazione dei nuovi utenti rimane impossibile, visto che la KVM potrebbe non disporre di un indirizzo pubblico raggiungibile, ma il login degli utenti può avvenire senza problemi, visto che avviene sull'handler raggiungibile solo dalla rete privata mesh.

11.2.2 Certificate-Authority

Un attaccante che ottenga il controllo della Certificate-Authority potrebbe firmare un certificato mTLS valido per qualunque `organization` (sezione 4.4) e impersonare così qualsiasi servizio interno, oppure rifiutarsi di rilasciare i certificati aggiornati per i servizi che richiedono un rinnovo.

Realisticamente, però, non può contattare nessuno a parte il broker MQTT, perché secondo le regole ACL di Headscale, può soltanto rispondere alle chiamate dei servizi interni, senza poter iniziare nuove comunicazioni (sezione 4.3).

Non intacca RU-05 o RU-06, visto che la CA non ha alcun ruolo nella cifratura dei backup (sezione 6.3) né nella derivazione delle chiavi delle credenziali (Capitolo 5). Resta garantito anche RU-08 che non ha alcun legame con la CA. Ciò che cade è l'autenticazione reciproca tra servizi. I servizi, però, essendo impossibilitati a rinnovare i loro certificati, non possono più comunicare, quindi una volta scaduti, l'intero sistema si blocca.

La mancata autenticazione tra i servizi può essere facilmente rilevata. Quando ciò accade, fintanto che le indagini sulla compromissione non si concludono, una nuova istanza della CA può essere riavviata in una KVM su Proxmox, idealmente su un nodo diverso, garantendo un isolamento maggiore di quello offerto da un LXC (sezione 10.3).

11.2.3 Database-Vault

Un attaccante che ottenga il controllo totale di Database-Vault ottiene il pepper e la master key, e con quest'ultima decifra ogni email salvata nel database. Tuttavia non ottiene la password con la stessa facilità. Per quello, l'attaccante ha due possibili strade: la prima è rubare il database e attaccarlo offline, al costo del tempo di calcolo di Argon2id (Tabella 5.2). Questo approccio normalmente richiede molto tempo e risorse computazionali considerevoli, diventando praticamente impraticabile. Senza il pepper a proteggerle però, le password deboli e molto comuni possono facilmente essere bucate con un attacco a dizionario, anche se protette da un algoritmo di hashing lento e costoso come Argon2id.

La seconda via è ben più subdola: l'attaccante può rimanere passivo e osservare le chiamate al Database-Vault. In questo modo non solo può leggere le credenziali dei nuovi utenti, ma, analizzando le richieste di login e seguendole fino al database, può identificare a quali hash corrispondono le password inviate dagli utenti. Questo richiede che il malintenzionato rimanga nascosto per il tempo necessario a ricevere le richieste di login da tutti gli utenti registratisi prima della sua infiltrazione.

RU-06 cade quindi per intero sull'email, e si riduce a un costo temporale per le password complesse e poco comuni.

Resta comunque garantito RU-05: Database-Vault non ha accesso al filesystem di Storage-Service, ha solo la facoltà di chiedergli di creare un account (Capitolo 4). Resta garantito anche RU-08, per lo stesso motivo.

In questo caso, anche se l'attacco attivo venisse rilevato, ormai non ci sarebbe nulla da fare; bisogna dare per scontato che il database sia già stato rubato. Se invece fosse stato l'attacco passivo ad essere rilevato, mentre si indaga sulla compromissione, si possono interrompere completamente i servizi di registrazione e login e concedere agli utenti già autenticati un tag di raggiungibilità verso Storage-

Service, più duraturo di quello da dodici ore che già possiedono (Capitolo 7). In questo modo, gli utenti loggati possono continuare a gestire i backup senza doversi riautenticare.

In entrambi i casi, però, sarebbe più sensato interrompere il servizio e dichiarare l'incidente.

11.2.4 Network-Manager

Un attaccante che ottenga il controllo di Network-Manager può assegnarsi raggiungibilità verso ogni altro servizio e verso gli utenti (Capitolo 7). Può anche annullare o prolungare i permessi di accesso verso Storage-Service (Codice 7.3). Può quindi bloccare e controllare l'intero traffico del sistema, senza però poterlo leggere e decifrare. RU-04 è quindi compromesso. Non può comunque accedere ai dati degli utenti o alle loro credenziali (Capitolo 5), quindi RU-05, RU-06 e RU-08 restano validi.

In caso di compromissione, mentre si indaga sulle cause, si può comunque interrompere Network-Manager e avviarne una nuova istanza in una KVM differente, idealmente su un nodo diverso.

11.2.5 Storage-Service

Un attaccante che ottenga il controllo di Storage-Service eredita il privilegio di root con cui il servizio deve girare (sezione 6.2), e con esso legge la sottodirectory di ogni utente direttamente dal filesystem host: qui cade RU-08.

Cade anche RU-04 perché l'attaccante può impedire ogni comunicazione con Storage-Service. Può persino cifrare, corrompere o eliminare i backup di tutti gli utenti, ma questa è una violazione di RU-09, dichiaratamente fuori scope per questo progetto. Resta comunque garantito RU-05 perché i dati sono cifrati lato client e non arrivano mai in chiaro su Storage-Service (sezione 6.3). Anche RU-06 rimane garantito, perché le credenziali di accesso non passano per Storage-Service.

In questo caso, quando viene rilevata la compromissione, non c'è altro da fare se non dichiarare l'incidente e interrompere il servizio.

12

Conclusioni

12.1 Risultati raggiunti

Il Capitolo 1 pone l'obiettivo di progettare, implementare e validare RAM-USB come servizio di backup remoto multi-utente conforme ai requisiti dell'SRS [9], affidandosi ai principi zero-trust e defense-in-depth per difendere il sistema da attacchi esterni.

I dieci requisiti utente elencati nel Capitolo 3 sono, con due sole eccezioni, completamente coperti da requisiti di sistema implementati e tracciati fino al codice e ai test. Un utente può registrarsi fornendo solo email, password e chiave pubblica SSH (RU-01), autenticarsi nelle sessioni successive (RU-02), caricare e recuperare backup cifrati lato client da qualunque luogo (RU-03, RU-07), farlo solo dopo essersi autenticato (RU-04), con la garanzia che il contenuto dei backup sia illeggibile anche da un amministratore (RU-05), in uno spazio isolato da quello degli altri utenti (RU-08). Un amministratore può osservare la salute e le prestazioni del sistema senza mai toccare i dati dei singoli utenti (RU-10).

RU-06 (Avere la garanzia che nessuno, amministratori inclusi, possa leggere le proprie credenziali di accesso) non è pienamente supportato a causa delle limitazioni teoriche imposte dalla scelta di un sistema di autenticazione con email e password e vale pertanto solo sulla password e solo quando questa si trovi già persistita nel database in forma di hash. L'unico modo per evitare che un amministratore possa intercettare le credenziali in transito, infatti, è non inviarle affatto al server, ad esempio utilizzando un protocollo zero-knowledge, non oggetto di questa tesi.

RU-09, la garanzia che nessuno possa modificare o cancellare un backup, è stata dichiarata fuori perimetro fin dalle prime versioni dell'SRS, a causa dei tempi stretti del progetto ed è ripresa nella prossima sezione.

Le cinque sequenze d'uso che attraversano il sistema, registrazione, login, backup, ripristino e consultazione delle metriche, sono state eseguite end-to-end sulla distribuzione reale su Proxmox, con gli stessi risultati riportati nel Capitolo 9.

12.2 Limiti noti

La SRS accetta esplicitamente quattro rischi come costo del rilascio nei tempi previsti [9]. Normalmente questi rischi non sarebbero accettabili in un vero sistema di backup, e la loro risoluzione è prevista negli

sviluppi futuri.

RISK-01 è RU-09 stesso. Resta un miglioramento desiderabile, non ancora progettato, implementato o testato.

RISK-02 riguarda la master key e il pepper: risiedono in una variabile d'ambiente, senza una procedura di backup (DV-F-18) né di rotazione (DV-F-19). La loro perdita comporterebbe la perdita irreversibile dell'accesso a tutte le credenziali cifrate.

RISK-03 riguarda il client: al momento non containerizzato con Docker. Il codice Go del servizio esiste ed è stato testato, ma la containerizzazione necessita di ulteriori ragionamenti e decisioni di design.

RISK-04 è la dipendenza circolare della risoluzione DNS di Network-Manager dal proprio stesso MagicDNS: se Network-Manager pubblicasse una policy ACL non funzionante, potrebbe perdere anche la capacità di risolvere l'hostname di Headscale di cui avrebbe bisogno per correggerla. In tal caso, spetta ad un amministratore risolvere manualmente il problema.

12.3 Sviluppi futuri

La sottosezione 5.2.1 misura che, sotto scelta uniforme, il vincolo di almeno tre categorie di carattere vale meno di un decimo di bit di entropia, mentre un carattere aggiuntivo ne vale 6,57: sotto quell'ipotesi la regola di composizione è ridondante, perché una password davvero casuale usa già più categorie da sé. Il suo scopo reale, però, non è aumentare l'entropia, ma vincolare la scelta di una persona, e lì il suo valore non è misurabile senza la distribuzione reale delle password scelte dagli utenti. Questa scelta di design è quindi al momento ancora aperta, in attesa di analisi più approfondite.

Oltre a DV-F-18 e DV-F-19, già discussi come RISK-02, restano da progettare, il limite di spazio di archiviazione dedicato ad ogni utente e la replica automatica dei backup su CephFS, entrambi rimandati nella SRS come miglioramenti desiderabili piuttosto che vincoli del progetto.

Altri sviluppi futuri, non ancora progettati né inseriti nell'SRS, sono la gestione del bilanciamento automatico del carico, la protezione dell'endpoint pubblico di Entry-Hub da attacchi alla disponibilità, magari imponendo un rate limiting su quell'endpoint e il miglioramento della scalabilità del sistema, magari sbloccando la geo-distribuzione del server.

A

Elenco completo dei requisiti

Le tabelle seguenti raccolgono i requisiti di sistema funzionali, non funzionali e di dominio, organizzati secondo la stessa struttura dell'SRS originale [9].

Il capitolo 3 discute per intero i requisiti utente, i quali sono gli unici a non essere inclusi nelle tabelle sottostanti.

A.1 Requisiti di sistema funzionali

A.1.1 Client

Tabella A.1: Requisiti funzionali del Client.

ID	Requisito
CL-F-01	Deve generare autonomamente una coppia di chiavi SSH (pubblica/privata) prima della registrazione, senza mai trasmettere la chiave privata ad alcun componente del sistema.
CL-F-02	Su comando dell'utente, deve inviare a Entry-Hub, durante la registrazione (`POST /api/users`), solo la chiave pubblica SSH, insieme a email e password.
CL-F-03	Su comando dell'utente, deve ripetere il login (`POST /api/login`) prima della scadenza del permesso ACL di dodici ore, per mantenere la continuità di accesso a Storage-Service.
CL-F-04	Deve configurare e avviare Tailscale usando la pre-auth key ricevuta nella risposta di registrazione, per unirsi alla rete mesh privata.
CL-F-05	Deve risolvere Storage-Service tramite MagicDNS sulla rete mesh, senza fare affidamento su indirizzi IP statici.
CL-F-06	Su comando dell'utente, deve invocare `restic backup` verso Storage-Service via SFTP sulla porta TCP 2222 (ST-F-03), autenticandosi con la chiave privata SSH generata in CL-F-01.

ID	Requisito
CL-F-07	Su comando dell'utente, deve invocare `restic restore` verso Storage-Service via SFTP sulla porta TCP 2222 (ST-F-03), usando lo stesso metodo di autenticazione di CL-F-06.
CL-F-08	Deve gestire i codici di errore HTTP restituiti da Entry-Hub (400/401/403/500/502/503/504) senza esporre dettagli interni del sistema all'utente finale.
CL-F-09	Su comando dell'utente, deve validare localmente email, password e, solo in fase di registrazione, la chiave pubblica SSH, usando le stesse regole imposte da Entry-Hub (EH-F-04 per la registrazione, EH-F-05 per il login), prima di inviare `POST /api/users` o `POST /api/login`; in caso di fallimento della validazione locale, non deve trasmettere la richiesta.
CL-F-10	Deve ricavare l'endpoint di login di Entry-Hub da un valore di configurazione distinto da quello di registrazione, e deve usare per il login un nome coperto dal certificato che Entry-Hub presenta.
CL-F-11	Deve fissare la chiave host SSH di Storage-Service: registra la chiave ricevuta nella risposta alla registrazione (ST-F-16) prima del primo backup, la presenta a `ssh` come host noto e interrompe la connessione se la chiave presentata differisce.

A.1.2 Entry-Hub

Tabella A.2: Requisiti funzionali di Entry-Hub.

ID	Requisito
EH-F-01	Deve esporre un endpoint HTTPS pubblico di health-check (`GET /api/health`), con certificati firmati dalla CA pubblica Let's Encrypt.
EH-F-02	Deve esporre un endpoint HTTPS pubblico per la registrazione degli utenti (`POST /api/users`), con certificati firmati dalla CA pubblica Let's Encrypt.
EH-F-03	Deve esporre un endpoint HTTPS (`POST /api/login`) per l'autenticazione degli utenti registrati, servito su un listener separato raggiungibile solo dalla rete mesh privata (non da internet, a differenza del listener di EH-F-01/EH-F-02), con lo stesso certificato firmato dalla CA pubblica Let's Encrypt presentato da EH-F-01/EH-F-02, non mTLS; cambia solo la raggiungibilità di rete.
EH-F-04	`/api/users` deve accettare JSON e validare: presenza dei campi email, password e chiave pubblica SSH; dimensione del payload entro un limite definito; assenza di campi aggiuntivi inattesi; formato dell'email (RFC 5322); lunghezza della password tra 8 e 128 caratteri; complessità della password (almeno 3 categorie di caratteri tra minuscole, maiuscole, cifre, simboli); correttezza formale della chiave pubblica SSH.

ID	Requisito
EH-F-05	`/api/login` deve accettare JSON e validare: presenza dei campi email e password; dimensione del payload entro un limite definito; assenza di campi aggiuntivi inattesi; formato dell'email (RFC 5322); lunghezza della password tra 8 e 128 caratteri; complessità della password (almeno 3 categorie di caratteri).
EH-F-06	In caso di fallimento della validazione deve: rispondere con HTTP 400 (Bad Request) senza specificare quale problema sia stato riscontrato; registrare nel log il problema trovato senza identificare l'utente; non inoltrare la richiesta ad alcun altro servizio interno.
EH-F-07	In caso di validazione riuscita deve: registrare nel log l'esito della validazione senza identificare l'utente; inoltrare la richiesta a Security-Switch via mTLS; verificare che il certificato provenga da un Security-Switch; verificare la validità del certificato X.509.
EH-F-08	Deve inoltrare all'utente la risposta ricevuta da Security-Switch.
EH-F-09	Deve mappare gli errori interni sui codici HTTP 400/401/500/502/503, restituendo al client messaggi sanificati e mantenendo i log dettagliati solo internamente.
EH-F-10	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (`metrics/Entry-Hub`), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
EH-F-11	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.

A.1.3 Security-Switch

Tabella A.3: Requisiti funzionali di Security-Switch.

ID	Requisito
SS-F-01	Deve accettare solo connessioni mTLS da client che soddisfino: campo `organization` uguale a `EntryHub`; certificato X.509 valido; accesso alla rete mesh privata.
SS-F-02	Deve ri-validare l'input ricevuto, indipendentemente dalla validazione già effettuata da Entry-Hub.
SS-F-03	In caso di fallimento della validazione deve: rispondere con HTTP 400 (Bad Request) senza specificare quale problema sia stato riscontrato; registrare nel log il problema trovato senza identificare l'utente; non inoltrare la richiesta ad alcun altro servizio interno.
SS-F-04	In caso di validazione riuscita deve: registrare nel log l'esito della validazione senza identificare l'utente; inoltrare la richiesta a Database-Vault via mTLS, verificando che: il certificato provenga da un Database-Vault; il certificato X.509 sia valido.

ID	Requisito
SS-F-05	Dopo la conferma di autenticazione riuscita da Database-Vault, deve richiedere a Network-Manager (via mTLS) di concedere a quell'utente l'accesso a Storage-Service per dodici ore.
SS-F-06	Deve mappare gli errori sui codici HTTP 400/401/403/500/502/504.
SS-F-07	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (`metrics/Security-Switch`), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
SS-F-08	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.
SS-F-09	Dopo la conferma di registrazione riuscita da Database-Vault, deve richiedere a Network-Manager (via mTLS) di creare un utente Headscale dedicato e generare una pre-auth key per il nuovo account, includendo poi quella chiave nella risposta a Entry-Hub.

A.1.4 Database-Vault

Tabella A.4: Requisiti funzionali di Database-Vault.

ID	Requisito
DV-F-01	Deve accettare solo connessioni mTLS da client che soddisfino: campo `organization` uguale a `SecuritySwitch`; certificato valido; accesso alla rete mesh privata.
DV-F-02	Deve ri-validare l'input ricevuto, indipendentemente dalla validazione già effettuata da Security-Switch.
DV-F-03	Deve calcolare l'hash SHA-256 dell'email per l'indicizzazione e come chiave primaria, senza mai registrare nel log l'email in chiaro.
DV-F-04	Deve cifrare l'email dell'utente: derivare una chiave di cifratura per record dalla master key con HKDF-SHA256 e un salt casuale a 16 byte, poi cifrare l'email con AES-256-GCM usando quella chiave derivata e un nonce casuale a 12 byte.
DV-F-05	La master key dovrebbe provenire da una sorgente configurabile, con validazione della lunghezza (32 byte).
DV-F-06	Deve mantenere un pepper come variabile d'ambiente.
DV-F-07	Deve calcolare l'hash della password con Argon2id: memoria 47104 KiB (46 MiB), 2 iterazioni, parallelismo 1, output a 32 byte, usando un salt casuale per record e il pepper (DV-F-06).
DV-F-08	Deve salvare il record utente in una transazione atomica.
DV-F-09	Deve richiedere a Storage-Service la creazione dell'utente POSIX univoco sul server, con username `user<xxxxxx>` (`xxxxxx`: 6 caratteri casuali da un alfabeto base-36), e attenderne la risposta.

ID	Requisito
DV-F-10	Se la creazione dell'utente POSIX fallisce, deve eliminare l'utente dal database e informare Security-Switch che la registrazione è fallita.
DV-F-11	Dopo aver creato il record utente e l'utente POSIX, deve informare Security-Switch che l'utente è stato registrato.
DV-F-12	Deve rifiutare (HTTP 409) le registrazioni con un'email o una chiave SSH già esistenti, senza fornire dettagli sull'errore.
DV-F-13	Durante il login, deve recuperare il salt associato all'email tramite l'hash SHA-256 dell'email (DV-F-03).
DV-F-14	Deve ricalcolare Argon2id sulla password ricevuta usando il salt recuperato e il pepper, confrontando il risultato con l'hash memorizzato.
DV-F-15	Deve rispondere con lo stesso codice HTTP 401 sia per un'email inesistente sia per una password errata, senza distinguere i due casi né nella risposta né nel log.
DV-F-16	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (`metrics/Database-Vault`), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
DV-F-17	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.
DV-F-18	Dovrebbe esistere una procedura di backup della master key.
DV-F-19	Dovrebbe esistere una procedura di rotazione della master key.
DV-F-20	In caso di fallimento della validazione deve: rispondere con HTTP 400 (Bad Request) senza specificare quale problema sia stato riscontrato; registrare nel log il problema trovato senza identificare l'utente; non inoltrare la richiesta ad alcun altro servizio interno.

A.1.5 Storage-Service

Tabella A.5: Requisiti funzionali di Storage-Service.

ID	Requisito
ST-F-01	Deve accettare connessioni mTLS solo da client che soddisfino: campo `organization` uguale a `DatabaseVault`; certificato valido; accesso alla rete mesh privata.
ST-F-02	Deve fornire upload/download di file già cifrati lato client, senza mai elaborare contenuto in chiaro.
ST-F-03	L'accesso deve avvenire esclusivamente via SFTP sulla porta TCP 2222, autenticato con la chiave pubblica SSH registrata dall'utente.
ST-F-04	Deve vietare esplicitamente qualsiasi forma di connessione SSH diversa da SFTP.
ST-F-05	Ogni utente deve disporre di uno spazio di storage isolato, non accessibile dagli altri utenti.

ID	Requisito
ST-F-06	Su richiesta di Database-Vault via mTLS, deve creare sul sistema un utente POSIX con username `user<xxxxxx>` (`xxxxxx`: 6 caratteri casuali da un alfabeto base-36).
ST-F-07	Deve garantire che l'utente POSIX non possa mai uscire dalla propria directory.
ST-F-08	L'account POSIX creato non deve avere una home directory tradizionale né una shell interattiva; l'unico spazio scrivibile è la sottodirectory dedicata all'interno del chroot.
ST-F-09	La configurazione sshd di Storage-Service deve avere `PasswordAuthentication no` e `PermitRootLogin no`, indipendentemente dal fatto che gli account creati non abbiano una password impostata.
ST-F-10	A seguito della creazione, riuscita o fallita, dell'utente POSIX, deve riportare l'esito a Database-Vault.
ST-F-11	Ad ogni tentativo di connessione SFTP dell'utente, deve recuperare la chiave pubblica corrente dell'utente da Database-Vault tramite `AuthorizedKeysCommand`.
ST-F-12	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (`metrics/Storage-Service`), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
ST-F-13	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.
ST-F-14	Dovrebbe imporre limiti di quota per utente.
ST-F-15	Dovrebbe garantire: replica automatica dei dati; tolleranza ai guasti per almeno un nodo; consistenza dei dati; possibilità di espansione tramite l'aggiunta di nuovi nodi senza interrompere il servizio.
ST-F-16	Deve restituire la propria chiave pubblica host SSH nella risposta alla richiesta di creazione dell'utente POSIX proveniente da Database-Vault (ST-F-10), affinché possa essere inoltrata al client e fissata da questo (CL-F-11).

A.1.6 Network-Manager

Tabella A.6: Requisiti funzionali di Network-Manager.

ID	Requisito
NM-F-01	Deve garantire che solo Entry-Hub possa contattare Security-Switch.
NM-F-02	Deve garantire che solo Security-Switch e Storage-Service possano contattare Database-Vault.
NM-F-03	Deve garantire che solo Security-Switch possa contattare Network-Manager.
NM-F-04	Deve garantire che tutti i componenti interni della rete, eccetto gli utenti, possano contattare la Certificate-Authority sulla rete mesh.
NM-F-05	Deve garantire che solo gli utenti autenticati possano vedere e contattare Storage-Service.

ID	Requisito
NM-F-06	Deve garantire che gli utenti registrati ma non autenticati possano vedere e contattare solo Entry-Hub.
NM-F-07	Deve garantire che gli utenti registrati e autenticati possano vedere e contattare solo Entry-Hub e Storage-Service.
NM-F-08	Su richiesta di Security-Switch, a seguito di una registrazione riuscita, deve creare un utente Headscale dedicato e generare una pre-auth key a breve durata per il nuovo account.
NM-F-09	Dopo un login riuscito, su richiesta di Security-Switch, deve assegnare al nodo dell'utente il tag ACL che abilita la raggiungibilità verso Storage-Service, registrando una scadenza a dodici ore da quel momento.
NM-F-10	Deve verificare periodicamente le scadenze registrate e rimuovere il tag ACL dai nodi scaduti, automaticamente e senza intervento manuale.
NM-F-11	La scadenza di ogni permesso deve essere persistita, non mantenuta solo in memoria, per non perdere lo stato in caso di riavvio di Network-Manager.
NM-F-12	La creazione di pre-auth key e la gestione dei tag ACL devono essere ristrette specificamente a Network-Manager, verificato via mutual TLS (campo `organization` uguale a `NetworkManager`).
NM-F-13	La pre-auth key serve unicamente a registrare il nodo come membro della mesh; da sola, non concede raggiungibilità verso Storage-Service.
NM-F-14	L'endpoint di coordinamento di Headscale è deliberatamente raggiungibile dalla rete pubblica (idealmente ospitato su un proprio VPS con indirizzo pubblico).
NM-F-15	Deve configurare MagicDNS con un dominio di base dedicato, in modo che Storage-Service sia risolvibile da tutti i nodi della mesh tramite un nome stabile anziché un IP.
NM-F-16	Il nodo mesh di Network-Manager deve accettare la configurazione DNS distribuita da Headscale (MagicDNS), in modo che le proprie chiamate in uscita verso Certificate-Authority e MQTT-Broker risolvano quegli hostname sulla mesh, anziché ricadere su un percorso di rete privo di equivalente in produzione.
NM-F-17	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (`metrics/Network-Manager`), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
NM-F-18	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.
NM-F-19	Deve configurare MagicDNS con un record che risolva il nome pubblico di Entry-Hub sul suo indirizzo mesh, così che un membro della rete che contatta il listener di login (EH-F-03) usi il nome coperto dal certificato di Entry-Hub.
NM-F-20	Deve garantire che solo i servizi che pubblicano metriche (Entry-Hub, Security-Switch, Database-Vault, Storage-Service, Network-Manager, Certificate-Authority) e Metrics-Collector possano contattare MQTT-Broker.
NM-F-21	Deve garantire che solo Grafana possa contattare Metrics-Collector.

A.1.7 Certificate-Authority

Tabella A.7: Requisiti funzionali di Certificate-Authority.

ID	Requisito
CA-F-01	Deve garantire che i componenti che presentano certificati per mTLS siano realmente chi dichiarano di essere.
CA-F-02	Deve garantire l'emissione, la rotazione, la revoca e la verifica dei certificati mTLS.
CA-F-03	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (`metrics/Certificate-Authority`), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
CA-F-04	Deve accettare un bootstrap token monouso, distribuito fuori banda a ciascun servizio, per l'emissione del certificato iniziale; i rinnovi successivi devono avvenire tramite il certificato mTLS corrente, non tramite il token.

A.1.8 Sistema di monitoraggio

Tabella A.8: Requisiti funzionali del sistema di monitoraggio (MQTT-Broker / Metrics-Collector / TimescaleDB / Grafana).

ID	Requisito
MT-F-01	Metrics-Collector può leggere solo `metrics/*`.
MT-F-02	Metrics-Collector deve scartare le metriche il cui campo `service` non corrisponde al topic da cui provengono.
MT-F-03	Le metriche devono essere memorizzate come hypertable TimescaleDB, con ritenzione automatica di 30 giorni e compressione dopo 7 giorni.
MT-F-04	Devono esistere dashboard Grafana per tempo di risposta, throughput e connessioni attive.

A.1.9 Infrastruttura di rete e PKI

Tabella A.9: Requisiti funzionali di rete e PKI.

ID	Requisito
PKI-F-01	Ogni servizio deve autenticarsi mutuamente con certificati X.509 emessi da una CA valida.
PKI-F-02	Ogni servizio deve verificare il campo `organization` del certificato, non solo la sua validità.
PKI-F-03	Dovrebbe esistere una procedura di rotazione e revoca dei certificati.

ID	Requisito
NET-F-01	La comunicazione tra servizi deve avvenire sulla rete privata; gli endpoint pubblici di Entry-Hub e l'endpoint di coordinamento Headscale di Network-Manager (NM-F-14) sono le uniche superfici deliberatamente esposte pubblicamente dal sistema.
NET-F-02	Il TLS utilizzato deve essere in versione 1.3.

A.2 Requisiti non funzionali

A.2.1 Di prodotto

Tabella A.10: Requisiti non funzionali di prodotto.

ID	Requisito
RNF-SEC-01	Nessun componente server-side accede mai al contenuto dei file di backup in chiaro, poiché la cifratura avviene lato client prima della trasmissione. Questo non si estende alle credenziali di accesso: email e password transitano, cifrate (TLS/mTLS), attraverso Entry-Hub, Security-Switch e Database-Vault per la validazione e l'hashing, pur senza mai essere persistite o registrate nei log in chiaro (cfr. DV-F-03, RD-01).
RNF-SEC-02	Zero-trust: nessun servizio deve fidarsi implicitamente dei dati ricevuti da un altro, anche se autenticato via mTLS.
RNF-SEC-03	Defense-in-depth: ogni livello ri-valida l'input in modo indipendente.
RNF-SEC-04	Tutta la comunicazione tra servizi deve usare mTLS, senza eccezioni.
RNF-REL-01	Il sistema deve tollerare la compromissione isolata di un singolo componente, senza che questa si propaghi agli altri.
RNF-PERF-01	La latenza delle richieste HTTP deve essere tracciata (p50/p95/p99).
RNF-USA-01	I messaggi di errore devono essere comprensibili e correttamente categorizzati per codice HTTP.
RNF-MAINT-01	Ogni servizio deve poter essere isolato, ri-certificato e riavviato individualmente senza impattare gli altri.

A.2.2 Organizzativi

Tabella A.11: Requisiti non funzionali organizzativi.

ID	Requisito
RNF-ORG-01	Linguaggio di implementazione: Go.
RNF-ORG-03	Licenza open-source MIT.
RNF-ORG-04	Piattaforma di distribuzione: Proxmox VE (KVM per Storage-Service, Database-Vault, Network-Manager; LXC per gli altri servizi).
RNF-ORG-05	Sviluppo ed esercizio garantiti su macOS e Linux (con Docker).

A.2.3 Esterni

Tabella A.12: Requisiti non funzionali esterni.

ID	Requisito
RNF-EXT-01	Poiché il sistema elabora dati personali (l'email), dovrebbe rispettare le normative sulla privacy applicabili (es. GDPR). Attualmente fuori perimetro.

A.3 Requisiti di dominio

Tabella A.13: Requisiti di dominio (RD).

ID	Requisito
RD-01	Qualsiasi nuovo componente introdotto in futuro non deve creare un percorso lungo il quale dati sensibili in chiaro attraversino o vengano registrati da un componente diverso dal client o dal componente strettamente necessario alla loro cifratura/decifratura.
RD-02	Le chiavi derivate (tramite HKDF) non devono mai essere persistite: qualsiasi nuovo requisito di memorizzazione di chiavi deve essere valutato rispetto a questo vincolo prima di essere accettato.
RD-03	Argon2id e AES-256-GCM sono vincoli tecnologici non negoziabili.
RD-04	Il principio di *fail-secure* si applica a ogni componente: in caso di incertezza sulla validità di una richiesta, il comportamento predefinito è negare l'accesso.

Bibliografia

[1] Francesco Verrengia. RAM-USB: creation and ownership of each user's chroot root and `data` directory.
`https://github.com/Verryx-02/RAM-USB/blob/3f20762d732f412fc1e724c41658a3d493a781a5/services/storage-service/internal/posixuser/creator.go#L157-L168`, 2026.

[2] Francesco Verrengia. RAM-USB: permission bits of each user's chroot root and `data` directory.
`https://github.com/Verryx-02/RAM-USB/blob/3f20762d732f412fc1e724c41658a3d493a781a5/services/storage-service/internal/posixuser/dirmaker.go#L36-L50`, 2026.

[3] David A. Patterson, Garth Gibson, e Randy H. Katz. A case for redundant arrays of inexpensive disks (RAID). In *Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data*, pp. 109–116, New York, NY, USA, 1988. ACM.
`https://doi.org/10.1145/50202.50214`.

[4] Riccardo Gottardi. GitHub profile.
`https://github.com/Riccardo-Gottardi`, 2026.

[5] Barry W. Boehm. A spiral model of software development and enhancement. *Computer*, 21(5):61–72, 1988.
A pagamento IEEE: `https://doi.org/10.1109/2.59`
Liberamente accessibile: `https://www.cse.msu.edu/~cse435/Homework/HW3/boehm.pdf`.

[6] Francesco Verrengia. RAM-USB contributing guidelines.
`https://github.com/Verryx-02/RAM-USB/blob/main/CONTRIBUTING.md`, 2026.

[7] Scott Rose, Oliver Borchert, Stu Mitchell, e Sean Connelly. NIST special publication 800-207: Zero Trust Architecture.
`https://doi.org/10.6028/NIST.SP.800-207`, 2020.

[8] National Institute of Standards and Technology. NIST special publication 800-53 revision 5: Security and privacy controls for information systems and organizations.
`https://doi.org/10.6028/NIST.SP.800-53r5`, 2020.

[9] Francesco Verrengia. RAM-USB software requirements specification.
`https://github.com/Verryx-02/RAM-USB/blob/main/docs/Software_Requirements_Specification.md`, 2026.

[10] Francesco Verrengia. RAM-USB software requirements specification, v1.0: original wording of NM-F-12.
`https://github.com/Verryx-02/RAM-USB/blob/3ef723e07a0292337fc48a1cd2e8c952f2f2e2ee/docs/Software_Requirements_Specification.md?plain=1#L260`, 2026.

[11] Francesco Verrengia. RAM-USB: correct NM-F-12 wording and document how headscale's CLI satisfies it.
`https://github.com/Verryx-02/RAM-USB/commit/975be1f467873628532a807e852f086dba29a9ef`, 2026.

[12] Francesco Verrengia. RAM-USB: rework NM-F-12/NM-F-14/NET-F-01 for a standalone public headscale and retire EH-F-12.
https://github.com/Verryx-02/RAM-USB/commit/f2d17f41668ae2c978c849ac308ac4fa52f7b409, 2026.

[13] Francesco Verrengia. RAM-USB contributing guidelines: Commit structure (section 2.2).
https://github.com/Verryx-02/RAM-USB/blob/7984a140e0ce7f6b3ea89470801c0d26ecbd4aa1/CONTRIBUTING.md?plain=1#L33-L101, 2026.

[14] Google. distroless: Language focused docker images, minus the operating system.
https://github.com/GoogleContainerTools/distroless, 2026.

[15] Murugiah Souppaya, John Morello, e Karen Scarfone. Application container security guide.
https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-190.pdf, 2017. NIST Special Publication 800-190.

[16] Michael Wager. Evaluating the security of containers through reduction of potentially vulnerable components.
https://www.mwager.de/assets/component_reduction_paper.pdf, 2024. Hochschule Augsburg.

[17] Josh Aas, Richard Barnes, Benton Case, Zakir Durumeric, Peter Eckersley, Alan Flores-López, J. Alex Halderman, Jacob Hoffman-Andrews, James Kasten, Eric Rescorla, Seth Schoen, e Brad Warren. Let's Encrypt: An automated certificate authority to encrypt the entire web. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19)*, pp. 2473–2487, New York, NY, USA, 2019. ACM. A pagamento (ACM Digital Library): https://doi.org/10.1145/3319535.3363192 Liberamente accessibile (ospitata da ISRG, l'organizzazione dietro Let's Encrypt): https://www.isrg.org/documents/letsencryptCCS2019.pdf.

[18] Francesco Verrengia. RAM-USB: decodeJSON and ValidateRegister.
https://github.com/Verryx-02/RAM-USB/blob/7c1e991f953a4c686e09f8142d0b3c1d9d64fc5f/pkg/validation/validation.go#L125-L173, 2026.

[19] Francesco Verrengia. RAM-USB: shared internal libraries.
https://github.com/Verryx-02/RAM-USB/tree/main/pkg, 2026.

[20] restic. restic: Fast, secure, efficient backup program.
https://github.com/restic/restic, 2026.

[21] Tatu Ylönen e Sami Lehtinen. SSH file transfer protocol.
https://datatracker.ietf.org/doc/html/draft-ietf-secsh-filexfer-02, 2001. la revisione implementata da OpenSSH, mai divenuta RFC.

[22] OWASP. Fail securely.
https://owasp.org/www-community/Fail_securely, 2026.

[23] Smallstep. certificates: A private certificate authority (x.509 & ssh) & acme server for secure automated certificate management.
https://github.com/smallstep/certificates, 2026.

[24] Juan Font. headscale: An open source, self-hosted implementation of the tailscale control server.
https://github.com/juanfont/headscale, 2026.

[25] Timescale. TimescaleDB: A time-series database for high-performance real-time analytics.
https://github.com/timescale/timescaledb, 2026.

[26] Grafana Labs. Grafana: The open and composable observability and data visualization platform. https://github.com/grafana/grafana, 2026.

[27] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Tesi di Dottorato di Ricerca, University of California, Irvine, 2000. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm.

[28] Peter W. Resnick. Internet message format. https://doi.org/10.17487/RFC5322, 2008. RFC 5322.

[29] Hugo Krawczyk e Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). https://doi.org/10.17487/RFC5869, 2010. RFC 5869.

[30] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. https://doi.org/10.6028/NIST.SP.800-38D, 2007. NIST Special Publication 800-38D.

[31] Francesco Verrengia. RAM-USB: derivazione della chiave per record con HKDF-SHA256. https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/encryption/encryption.go#L88-L109, 2026.

[32] Alex Biryukov, Daniel Dinu, Dmitry Khovratovich, e Simon Josefsson. Argon2 memory-hard function for password hashing and proof-of-work applications. https://doi.org/10.17487/RFC9106, 2021. RFC 9106.

[33] OWASP. Password storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, 2026.

[34] Francesco Verrengia. RAM-USB: salvataggio del record utente in transazione atomica. https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/storage/storage.go#L111-L146, 2026.

[35] Francesco Verrengia. RAM-USB: rollback compensativo dopo un fallimento di storage-service. https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/registration/registration.go#L157-L165, 2026.

[36] Francesco Verrengia. RAM-USB: esito unico del login per email inesistente e password errata. https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/login/login.go#L160-L185, 2026.

[37] Francesco Verrengia. RAM-USB: verifica fittizia che pareggia i tempi di risposta del login. https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/password/hash.go#L178-L193, 2026.

[38] Francesco Verrengia. RAM-USB: generation and local storage of the Restic repository password. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/user-client/internal/reposecret/reposecret.go#L51-L77, 2026.

[39] restic. restic documentation: references and repository design. https://restic.readthedocs.io/en/stable/100_references.html, 2026.

[40] OpenBSD Project. sshd_config(5): Openssh daemon configuration file. https://man.openbsd.org/sshd_config.5, 2026.

[41] Francesco Verrengia. RAM-USB: Storage-service hardened `sshd_config`. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/deployments/docker/storage-service/sshd_config, 2026.

[42] Francesco Verrengia. RAM-USB: fail-secure exit paths of the `AuthorizedKeysCommand` binary. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/storage-service/cmd/authorized-keys-command/main.go#L97-L132, 2026.

[43] Francesco Verrengia. RAM-USB: Restic repository URL and SFTP transport command. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/user-client/internal/restic/restic.go#L74-L90, 2026.

[44] Francesco Verrengia. RAM-USB: mesh reachability rules published to Headscale. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/headscale/policy.go#L198-L336, 2026.

[45] Francesco Verrengia. RAM-USB: mesh identity and pre-auth key creation at registration. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/headscale/client.go#L198-L217, 2026.

[46] Francesco Verrengia. RAM-USB: dodici ore fisse di durata del grant e assegnazione del tag di accesso a Storage-Service dopo il login. Durata del grant: https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/headscale/client.go#L111-L116 Assegnazione del tag di accesso: https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/headscale/client.go#L251-L275, 2026.

[47] Francesco Verrengia. RAM-USB: SQLite schema of the permanent email-to-pre-auth-key-ID mapping. https://github.com/Verryx-02/RAM-USB/blob/cd4b9fcaf3b1f503d96040287d4849c8cc17f2ac/services/network-manager/internal/grants/meshusers.go#L20-L25, 2026.

[48] Francesco Verrengia. RAM-USB: SQLite schema of the persisted access-grants table. https://github.com/Verryx-02/RAM-USB/blob/cd4b9fcaf3b1f503d96040287d4849c8cc17f2ac/services/network-manager/internal/grants/store.go#L87-L94, 2026.

[49] Francesco Verrengia. RAM-USB: periodic revocation of expired access grants. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/grants/sweep.go#L54-L88, 2026.

[50] Headscale. Headscale FAQ: can I use headscale and tailscale on the same machine? https://headscale.net/stable/about/faq/, 2026.

[51] Francesco Verrengia. RAM-USB: reverse proxy che separa il percorso amministrativo, con mTLS, da quello di coordinamento, senza controlli. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/deployments/docker/headscale/nginx.conf#L110-L139, 2026.

[52] ICANN. Available pool of unallocated IPv4 internet addresses now completely emptied. https://itp.cdn.icann.org/en/files/announcements/release-03feb11-en.pdf, 2011. Comunicato stampa, 3 febbraio 2011.

[53] Jason Weil, Victor Kuarsingh, Chris Donley, Christopher Liljenstolpe, e Marla Azinger. IANA-reserved IPv4 prefix for shared address space. https://doi.org/10.17487/RFC6598, 2012. RFC 6598, BCP 153.

[54] Simon Perreault, Ikuhei Yamagata, Shin Miyakawa, Akira Nakagawa, e Hiroyuki Ashida. Common requirements for carrier-grade NATs (CGNs).
https://doi.org/10.17487/RFC6888, 2013. RFC 6888, BCP 127.

[55] Mat Ford, Mohamed Boucadair, Alain Durand, Pierre Levis, e Phil Roberts. Issues with IP address sharing.
https://doi.org/10.17487/RFC6269, 2011. RFC 6269.

[56] Eclipse Foundation. Eclipse Mosquitto: an open source MQTT broker.
https://github.com/eclipse-mosquitto/mosquitto, 2026.

[57] Francesco Verrengia. RAM-USB: shape of the aggregated metrics payload.
https://github.com/Verryx-02/RAM-USB/blob/dd0302834b0222ec9ef1899a01735d16725c5137/pkg/metrics/payload.go#L50-L57, 2026.

[58] Andrew Banks e Rahul Gupta. MQTT version 3.1.1 plus errata 01.
https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/errata01/os/mqtt-v3.1.1-errata01-os-complete.html, 2015.

[59] Francesco Verrengia. RAM-USB: hypertable, retention, and compression policy for the metrics table.
https://github.com/Verryx-02/RAM-USB/blob/17dffbd13729426591f7052f8e2b782d41332d67/services/metrics-collector/migrations/000001_create_metrics_table.up.sql#L56-L76, 2026.

[60] Francesco Verrengia. RAM-USB: test che email inesistente e password errata producano lo stesso esito.
https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/services/database-vault/internal/login/login_test.go#L88-L129, 2026.

[61] golangci-lint. golangci-lint: analizzatore statico aggregato per Go.
https://golangci-lint.run/, 2026.

[62] securego. gosec: analizzatore di sicurezza statico per Go.
https://github.com/securego/gosec, 2026.

[63] timakin. bodyclose: rilevamento di corpi di risposta HTTP non chiusi.
https://github.com/timakin/bodyclose, 2026.

[64] ryanrolds. sqlclosecheck: rilevamento di righe e statement SQL non chiusi.
https://github.com/ryanrolds/sqlclosecheck, 2026.

[65] kkHAIKE. contextcheck: verifica della propagazione del context.Context.
https://github.com/kkHAIKE/contextcheck, 2026.

[66] polyfloyd. errorlint: verifica delle convenzioni di wrapping degli errori in Go.
https://codeberg.org/polyfloyd/go-errorlint, 2026.

[67] Go Team. govulncheck: analisi delle vulnerabilità raggiungibili dal codice.
https://pkg.go.dev/golang.org/x/vuln/cmd/govulncheck, 2026.

[68] Gitleaks. gitleaks: rilevamento di segreti nei repository.
https://github.com/gitleaks/gitleaks, 2026.

[69] Aqua Security. trivy: scanner di vulnerabilità per immagini e filesystem.
https://github.com/aquasecurity/trivy, 2026.

[70] Francesco Verrengia. RAM-USB known issues (scratch cache).
https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/docs/Known_Issues.md, 2026.

[71] just-containers. s6-overlay: process supervisor for Docker containers, built on s6.
https://github.com/just-containers/s6-overlay, 2026.

[72] Francesco Verrengia. RAM-USB: Storage-service's s6-overlay service directories.
https://github.com/Verryx-02/RAM-USB/tree/b7b2914230f844745df007057315c19f57d1a576/deployments/docker/storage-service/rootfs/etc/s6-overlay/s6-rc.d, 2026.

[73] Francesco Verrengia. RAM-USB: Entry-hub's distroless runtime stage.
https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/deployments/docker/entry-hub/Dockerfile#L104-L111, 2026.

[74] Proxmox Server Solutions GmbH. Proxmox VE: open-source server virtualization (KVM and LXC).
https://www.proxmox.com/en/proxmox-virtual-environment/overview, 2026.

[75] Docker Inc. What is docker?
https://docs.docker.com/get-started/docker-overview/, 2026.

[76] Francesco Verrengia. RAM-USB: Storage-service Proxmox deployment notes.
https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/deployments/proxmox/storage-service.md, 2026.

[77] Francesco Verrengia. RAM-USB: one Docker Compose file per service.
https://github.com/Verryx-02/RAM-USB/tree/b7b2914230f844745df007057315c19f57d1a576/deployments/compose, 2026.

[78] Proxmox Server Solutions GmbH. qm: live migration delle macchine virtuali KVM.
https://git.proxmox.com/?p=pve-docs.git;a=blob_plain;f=qm.adoc;hb=HEAD, 2026.

[79] Proxmox Server Solutions GmbH. pct: migrazione dei container LXC, nessuna migrazione a caldo disponibile.
https://git.proxmox.com/?p=pve-docs.git;a=blob_plain;f=pct.adoc;hb=HEAD, 2026.

[80] Proxmox Server Solutions GmbH. Cluster manager: requisito di latenza sotto i 5ms tra i nodi del cluster.
https://pve.proxmox.com/pve-docs/chapter-pvecm.html, 2026.